Implementação do Marco 3

https://gemini.google.com/app/c9876070cadee863

User prompt: Passo 2 dos três Developers O próximo marco deve continuar pequeno. Não vamos começar a Fase 2 ainda. O objetivo será: Transformar a Fase 1 de uma demonstração com um computador em uma fase curta completa, contendo três situações interativas, progresso 0/3 → 3/3, conclusão da fase, salvamento e uma tela de resultado. Ao final, o fluxo deverá ser: MENU ↓ PERFIL ↓ FASE 1 — LABORATÓRIO ↓ 3 SITUAÇÕES INTERATIVAS ↓ 0/3 → 1/3 → 2/3 → 3/3 ↓ FASE CONCLUÍDA ↓ TELA DE RESULTADO ↓ PROGRESSO SALVO 1. Reunião curta entre os três Antes de programar, definam oficialmente as três situações da Fase 1. A primeira já existe: 1. Computador ligado Acrescentem somente mais duas: 2. Lixo deixado no laboratório 3. Equipamento/material utilizado de forma inadequada Cada situação deverá possuir duas opções simples. Computador Já existente: Desligar corretamente → positiva Ignorar → negativa Lixo Sugestão: Jogar o lixo na lixeira → positiva Deixar o lixo no chão → negativa Equipamento Sugestão: Guardar/utilizar corretamente → positiva Usar de forma inadequada → negativa Não criem mais situações nesse momento. 2. Developer de Dados O trabalho dele agora é pequeno porque a estrutura principal já existe. Passo 2.1 — revisar o perfil atual O perfil já contém: pontuacao progresso escolhasRealizadas Isso provavelmente já é suficiente. Evitem criar outro banco ou outra estrutura paralela. Passo 2.2 — garantir que a conclusão da Fase 1 seja salva Quando a fase terminar, o perfil deverá ficar aproximadamente assim: progresso: { faseAtual: 2, fasesConcluidas: [1] } Isso já é compatível com o ProgressionService. O Developer de Dados deve garantir que, depois de: ProgressionService.concluirFase(perfil, 1) o resultado seja enviado para: salvarPerfil(perfil) e sobreviva ao reload. Passo 2.3 — não salvar estado visual desnecessário Não precisa salvar coisas como: posição X do jogador; posição Y; janela de diálogo aberta; qual direção o personagem estava olhando. Neste momento salvem somente dados importantes para o progresso. Passo 2.4 — criar teste de persistência da conclusão Testar: Perfil começa: faseAtual = 1 fasesConcluidas = [] Conclui Fase 1. Resultado: faseAtual = 2 fasesConcluidas = [1] Recarrega. Resultado continua: faseAtual = 2 fasesConcluidas = [1] Entrega do Developer de Dados Ele deverá conseguir afirmar: "Além de pontuação e escolhas, agora a conclusão da Fase 1 também permanece salva." 3. Developer de Back-end/Lógica Esse Developer será responsável principalmente pelas novas escolhas e pela regra de conclusão. Passo 3.1 — ampliar choicesData.js Hoje existem: desligar_computador ignorar_computador Adicionar as duas novas interações. Por exemplo: recolher_lixo deixar_lixo ambas com: interacaoId: "lixo_01" Depois: guardar_equipamento usar_equipamento_incorretamente com: interacaoId: "equipamento_01" Cada interação deve continuar sendo mutuamente exclusiva. 4. Definir pontuação Mantenham números simples. Exemplo: Computador: Desligar corretamente → +10 cuidado / +8 conduta Ignorar → 0 cuidado / -5 conduta Lixo: Recolher → +10 cuidado / +8 conduta Deixar → 0 cuidado / -5 conduta Equipamento: Usar/guardar certo → +10 cuidado / +8 conduta Usar errado → 0 cuidado / -8 conduta Esses valores ainda podem ser ajustados futuramente. O importante agora é o sistema funcionar. 5. Reutilizar ChoiceService Não criem: ComputerChoiceService TrashChoiceService EquipmentChoiceService Isso seria duplicação desnecessária. O mesmo: ChoiceService.registrarEscolha() deve processar as seis alternativas. Assim vocês comprovam que a arquitetura criada no Marco 1 é reutilizável. 6. Criar regra para contar interações resolvidas O Back-end deve fornecer uma forma simples de saber: computador_01 → resolvido? lixo_01 → resolvido? equipamento_01 → resolvido? Pode aproveitar: ChoiceService.interacaoConcluida() Então a Fase 1 poderá calcular: 0/3 1/3 2/3 3/3 Evitem fazer essa regra espalhada por vários objetos visuais. 7. Condição para concluir a Fase 1 A regra será extremamente clara: A Fase 1 termina quando as três interações obrigatórias forem resolvidas. Não importa se as respostas foram positivas ou negativas. Isso é importante. O jogador pode tomar decisões ruins e ainda terminar a fase. As escolhas influenciam a pontuação e posteriormente as consequências, não impedem obrigatoriamente a campanha. 8. Developer de Front-end — reorganizar a Fase 1 Agora o front-end deve transformar o laboratório atual em um pequeno cenário com três pontos. Algo aproximadamente assim:

*┌──────────────────────────────┐ LABORATÓRIO 🗑 LIXO 💻 COMPUTADOR 👤 JOGADOR ⚙ EQUIPAMENTO └──────────────────────────────┘ Ainda podem usar placeholders. Não é necessário produzir o mapa definitivo. 9. Reutilizar o sistema de interação O sistema atual: aproximar ↓ [E] Interagir ↓ diálogo ↓ escolha deverá funcionar nos três objetos. Não façam três sistemas diferentes. O objetivo deste marco é justamente demonstrar reutilização. 10. Melhorar a organização dos objetos interativos Neste ponto já vale evitar colocar tudo diretamente dentro de Stage1Scene. Podem criar algo como: src/entities/InteractiveObject.js ou uma pequena estrutura semelhante. Ela poderá guardar: id x y interacaoId titulo texto opcoes Mas mantenham simples. Não construam um motor genérico gigantesco. 11. Lixo interativo Quando o jogador se aproximar: [E] Interagir Depois: Há lixo no chão do laboratório. O que deseja fazer? [Colocar na lixeira] [Deixar onde está] A opção escolhida deverá passar pelo ChoiceService. 12. Equipamento interativo Mesma lógica: [E] Interagir Depois: Um equipamento foi deixado fora do lugar. O que deseja fazer? [Guardar corretamente] [Usar/deixar de forma inadequada] Novamente: ChoiceService.registrarEscolha(...) 13. Feedback visual após cada interação Depois de uma escolha: Boa escolha! +10 cuidado +8 conduta ou: Essa atitude prejudica o cuidado com o laboratório. Conduta -5 E o objeto deve visualmente indicar que já foi resolvido. Por exemplo: COMPUTADOR ✓ LIXO ✓ EQUIPAMENTO ✓ Não dependam somente de cor. 14. Atualizar o HUD Hoje existe: Objetivo: verifique o computador Cuidado: 0 Conduta: 0 Agora deverá se tornar algo como: Objetivo: cuide do laboratório Progresso: 1/3 Cuidado: 10 Conduta: 8 Após outra interação: Progresso: 2/3 E finalmente: Progresso: 3/3 15. Conclusão da fase Quando chegar a: 3/3 não mude imediatamente de tela. Mostre primeiro algo como: FASE CONCLUÍDA Você analisou todas as situações do laboratório. [FINALIZAR FASE] Nesse momento: ProgressionService.concluirFase(perfil, 1) e depois: salvarPerfil(perfil) 16. Criar StageResultScene Agora sim teremos uma nova cena. src/scenes/StageResultScene.js Ela deverá mostrar algo simples: FASE 1 CONCLUÍDA Perfil: João Cuidado: 20 Conduta: 11 Situações analisadas: 3/3 [CONTINUAR] [MENU PRINCIPAL] Não precisa calcular finais. Não precisa fazer ranking. Não precisa comparar jogadores. 17. O botão Continuar da tela de resultado Como a Fase 2 ainda não existe, não iniciem Fase 2. O botão pode levar inicialmente a uma tela simples ou ao menu e informar: Fase 2 desbloqueada. Será implementada no próximo marco. Ou simplesmente voltar ao menu. O importante é o perfil já possuir: faseAtual = 2 fasesConcluidas = [1] 18. Testes conjuntos Os três deverão executar este roteiro. Caminho positivo 1. Criar perfil João. 2. Entrar na Fase 1. 3. Resolver computador positivamente. 4. HUD → 1/3. 5. Resolver lixo positivamente. 6. HUD → 2/3. 7. Resolver equipamento positivamente. 8. HUD → 3/3. 9. Concluir fase. 10. Abrir StageResultScene. 11. Voltar ao menu. 12. Recarregar. 13. Confirmar: faseAtual = 2 fasesConcluidas = [1] pontuação mantida. Caminho misto Criar outro perfil. Por exemplo: Computador → positivo Lixo → negativo Equipamento → negativo A fase também deve chegar a: 3/3 mas com pontuação diferente. 19. Testar independência entre perfis Muito importante agora que existem vários perfis. Criar: João Maria João conclui a fase. Maria não. Depois selecionar Maria. O jogo não pode mostrar as escolhas ou a pontuação de João. 20. O que NÃO fazer no Passo 2 Ainda não façam: Fase 2 jogável; Fase 3; sistema de finais; banco SQL; servidor; conta online; autenticação; inventário; combate; mapa enorme; animações complexas; sistema completo de áudio; NPCs complexos; ranking online. O objetivo continua sendo incremental. Critério final do Marco 2 Só consideraremos o segundo passo concluído quando o fluxo funcionar assim: FASE 1 │ ├── COMPUTADOR │ ├── escolha positiva │ └── escolha negativa │ ├── LIXO │ ├── escolha positiva │ └── escolha negativa │ └── EQUIPAMENTO ├── escolha positiva └── escolha negativa ↓ 3 / 3 ↓ FASE CONCLUÍDA ↓ ProgressionService ↓ salvarPerfil() ↓ StageResultScene ↓ faseAtual = 2 fasesConcluidas = [1] choicesData.js export const CHOICES = { desligar_computador: { id: "desligar_computador", interacaoId: "computador_01", tipo: "positiva", cuidado: 10, conduta: 8, mensagem: "Você desligou corretamente o equipamento." }, ignorar_computador: { id: "ignorar_computador", interacaoId: "computador_01", tipo: "negativa", cuidado: 0, conduta: -5, mensagem: "O equipamento continuou ligado consumindo energia sem necessidade." } }; ChoiceService.js import { CHOICES } from '../data/choicesData.js'; export class ChoiceService { static interacaoConcluida(perfil, interacaoId) { if (!perfil || !Array.isArray(perfil.escolhasRealizadas)) { return false; } return perfil.escolhasRealizadas.some((registro) => { const choiceId = typeof registro === "string" ? registro : registro?.choiceId; return CHOICES[choiceId]?.interacaoId === interacaoId; }); } /** * Processa a escolha feita pelo jogador e atualiza os dados do perfil. * @param {Object} perfil Objeto do perfil atual * @param {string} choiceId Identificador da escolha feita * @returns {Object} Resultado do processamento */ static registrarEscolha(perfil, choiceId) { const escolha = CHOICES[choiceId]; if (!escolha || !perfil || typeof perfil !== "object") { return { sucesso: false, erro: "Escolha inválida ou não encontrada." }; } // Inicializa a lista de escolhas no perfil se ainda não existir if (!Array.isArray(perfil.escolhasRealizadas)) { perfil.escolhasRealizadas = []; } if (!perfil.pontuacao) { perfil.pontuacao = {}; } // Regra de Negócio: Impedir escolhas duplicadas const escolhaJaRegistrada = perfil.escolhasRealizadas.some((registro) => { const idRegistrado = typeof registro === "string" ? registro : registro?.choiceId; return idRegistrado === choiceId; }); if (escolhaJaRegistrada) { return { sucesso: false, erro: "Esta ação já foi realizada anteriormente.", perfil }; } // As alternativas do computador pertencem à mesma interação. if (escolha.interacaoId && this.interacaoConcluida(perfil, escolha.interacaoId)) { return { sucesso: false, erro: "Esta interação já foi resolvida anteriormente.", interacaoConcluida: true, perfil }; } // Aplicação das consequências na pontuação perfil.pontuacao.cuidado = (perfil.pontuacao.cuidado || 0) + escolha.cuidado; perfil.pontuacao.conduta = (perfil.pontuacao.conduta || 0) + escolha.conduta; // Registrar ação concluída perfil.escolhasRealizadas.push(choiceId); return { sucesso: true, mensagem: escolha.mensagem, impacto: { cuidado: escolha.cuidado, conduta: escolha.conduta }, perfil }; } } Faça apenas a parte do back-end/lógica*

Response: **src/data/choicesData.js**

```
export const CHOICES = {
  // Interação 1: Computador
  desligar_computador: {
    id: "desligar_computador",
    interacaoId: "computador_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    mensagem: "Você desligou corretamente o equipamento."
  },
  ignorar_computador: {
    id: "ignorar_computador",
    interacaoId: "computador_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -5,
    mensagem: "O equipamento continuou ligado consumindo energia sem necessidade."
  },

  // Interação 2: Lixo
  recolher_lixo: {
    id: "recolher_lixo",
    interacaoId: "lixo_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    mensagem: "Você recolheu o lixo e manteve o ambiente limpo."
  },
  deixar_lixo: {
    id: "deixar_lixo",
    interacaoId: "lixo_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -5,
    mensagem: "O lixo foi deixado no chão do laboratório."
  },

  // Interação 3: Equipamento
  guardar_equipamento: {
    id: "guardar_equipamento",
    interacaoId: "equipamento_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    mensagem: "Você guardou o equipamento no local adequado."
  },
  usar_equipamento_incorretamente: {
    id: "usar_equipamento_incorretamente",
    interacaoId: "equipamento_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -8,
    mensagem: "O equipamento foi utilizado de forma inadequada."
  }
};
```

src/services/ChoiceService.js

```
import { CHOICES } from '../data/choicesData.js';

export class ChoiceService {
  /**
   * Verifica se uma interação específica já foi resolvida no perfil.
   */
```

```
  static interacaoConcluida(perfil, interacaoId) {
    if (!perfil || !Array.isArray(perfil.escolhasRealizadas)) {
      return false;
    }

    return perfil.escolhasRealizadas.some((registro) => {
      const choiceId = typeof registro === "string" ? registro : registro?.choiceId;
      return CHOICES[choiceId]?.interacaoId === interacaoId;
    });
  }

  /**
   * Retorna a contagem de interações resolvidas de uma lista esperada na fase.
   * Exemplo de retorno: { resolvidas: 2, total: 3, progressoTexto: "2/3" }
   */
  static obterProgressoInteracoes(perfil, listaInteracoes = []) {
    if (!Array.isArray(listaInteracoes) || listaInteracoes.length === 0) {
      return { resolvidas: 0, total: 0, progressoTexto: "0/0" };
    }

    const resolvidas = listaInteracoes.filter((interacaoId) =>
      this.interacaoConcluida(perfil, interacaoId)
    ).length;

    return {
      resolvidas,
      total: listaInteracoes.length,
      progressoTexto: `${resolvidas}/${listaInteracoes.length}`
    };
  }

  /**
   * Verifica se todas as interações obrigatórias da fase foram concluídas.
   */
  static todasInteracoesConcluidas(perfil, listaInteracoes = []) {
    if (!Array.isArray(listaInteracoes) || listaInteracoes.length === 0) {
      return false;
    }

    return listaInteracoes.every((interacaoId) =>
      this.interacaoConcluida(perfil, interacaoId)
    );
  }

  /**
   * Processa a escolha feita pelo jogador e atualiza os dados do perfil.
   */
  static registrarEscolha(perfil, choiceId) {
    const escolha = CHOICES[choiceId];

    if (!escolha || !perfil || typeof perfil !== "object") {
      return { sucesso: false, erro: "Escolha inválida ou não encontrada." };
    }

    if (!Array.isArray(perfil.escolhasRealizadas)) {
      perfil.escolhasRealizadas = [];
    }

    if (!perfil.pontuacao) {
      perfil.pontuacao = { cuidado: 0, conduta: 0 };
    }

    // Valida duplicidade da escolha exata
    const escolhaJaRegistrada = perfil.escolhasRealizadas.some((registro) => {
      const idRegistrado = typeof registro === "string" ? registro : registro?.choiceId;
      return idRegistrado === choiceId;
    });

    if (escolhaJaRegistrada) {
      return {
        sucesso: false,
        erro: "Esta ação já foi realizada anteriormente.",
        perfil
      };
    }

    // Valida se a interação associada já foi resolvida por outra opção
    if (escolha.interacaoId && this.interacaoConcluida(perfil, escolha.interacaoId)) {
      return {
        sucesso: false,
        erro: "Esta interação já foi resolvida anteriormente.",
        interacaoConcluida: true,
        perfil
      };
    }

    // Atualização de pontos
    perfil.pontuacao.cuidado = (perfil.pontuacao.cuidado || 0) + escolha.cuidado;
    perfil.pontuacao.conduta = (perfil.pontuacao.conduta || 0) + escolha.conduta;

    // Registro no histórico do perfil
    perfil.escolhasRealizadas.push(choiceId);

    return {
      sucesso: true,
      mensagem: escolha.mensagem,
      impacto: { cuidado: escolha.cuidado, conduta: escolha.conduta },
      perfil
    };
  }
}
```

src/services/ProgressionService.js

```
export class ProgressionService {
  /**
   * Conclui a fase informada e atualiza a progressão do perfil para a próxima.
   */
  static concluirFase(perfil, faseId) {
    if (!perfil || typeof perfil !== "object") {
      return { sucesso: false, erro: "Perfil inválido." };
    }

    if (!perfil.progresso) {
      perfil.progresso = {
        faseAtual: 1,
        fasesConcluidas: []
      };
    }

    if (!Array.isArray(perfil.progresso.fasesConcluidas)) {
      perfil.progresso.fasesConcluidas = [];
    }

    // Registra a fase concluída sem duplicar
    if (!perfil.progresso.fasesConcluidas.includes(faseId)) {
      perfil.progresso.fasesConcluidas.push(faseId);
    }

    // Define a próxima fase
```

```
    perfil.progresso.faseAtual = Math.max(perfil.progresso.faseAtual, faseId + 1);

    return {
      sucesso: true,
      perfil
    };
  }
}
```

Validação das regras de negócio atendidas

- **Isolamento de Estado:** Todo o cálculo de progresso e regras de pontuação baseiam-se estritamente na instância do objeto `perfil` recebida, garantindo independência entre múltiplos perfis (ex: João e Maria).
- **Contagem 0/3 a 3/3:** O método `ChoiceService.obterProgressoInteracoes(perfil, ['computador_01', 'lixo_01', 'equipamento_01'])` centraliza o cálculo de progresso para a interface sem espalhar lógica nos componentes do Front-end.
- **Independência do resultado:** A verificação com `todasInteracoesConcluidas` aceita escolhas positivas ou negativas indiferentemente, exigindo apenas que todas as 3 interações tenham sido respondidas.

---

*User prompt: src/services/ChoiceService.js import { CHOICES } from '../data/choicesData.js'; export class ChoiceService { static interacaoConcluida(perfil, interacaoId) { if (!perfil || !Array.isArray(perfil.escolhasRealizadas)) { return false; } return perfil.escolhasRealizadas.some((registro) => { const choiceId = typeof registro === "string" ? registro : registro?.choiceId; return CHOICES[choiceId]?.interacaoId === interacaoId; }); } /** * Processa a escolha feita pelo jogador e atualiza os dados do perfil. * @param {Object} perfil Objeto do perfil atual * @param {string} choiceId Identificador da escolha feita * @returns {Object} Resultado do processamento */ static registrarEscolha(perfil, choiceId) { const escolha = CHOICES[choiceId]; if (!escolha || !perfil || typeof perfil !== "object") { return { sucesso: false, erro: "Escolha inválida ou não encontrada." }; } // Inicializa a lista de escolhas no perfil se ainda não existir if (!Array.isArray(perfil.escolhasRealizadas)) { perfil.escolhasRealizadas = []; } if (!perfil.pontuacao) { perfil.pontuacao = {}; } // Regra de Negócio: Impedir escolhas duplicadas const escolhaJaRegistrada = perfil.escolhasRealizadas.some((registro) => { const idRegistrado = typeof registro === "string" ? registro : registro?.choiceId; return idRegistrado === choiceId; }); if (escolhaJaRegistrada) { return { sucesso: false, erro: "Esta ação já foi realizada anteriormente.", perfil }; } // As alternativas do computador pertencem à mesma interação. if (escolha.interacaoId && this.interacaoConcluida(perfil, escolha.interacaoId)) { return { sucesso: false, erro: "Esta interação já foi resolvida anteriormente.", interacaoConcluida: true, perfil }; } // Aplicação das consequências na pontuação perfil.pontuacao.cuidado = (perfil.pontuacao.cuidado || 0) + escolha.cuidado; perfil.pontuacao.conduta = (perfil.pontuacao.conduta || 0) + escolha.conduta; // Registrar ação concluída perfil.escolhasRealizadas.push(choiceId); return { sucesso: true, mensagem: escolha.mensagem, impacto: { cuidado: escolha.cuidado, conduta: escolha.conduta }, perfil }; } } src/services/ProgressionService.js export class ProgressionService { /** * Conclui a fase atual e atualiza a fase ativa no perfil. * @param {Object} perfil Objeto do perfil atual * @param {number} faseConcluida Número da fase * @returns {Object} Perfil atualizado */ static concluirFase(perfil, faseConcluida) { if (!perfil.progresso) { perfil.progresso = { faseAtual: 1, fasesConcluidas: [] }; } if (!Array.isArray(perfil.progresso.fasesConcluidas)) { perfil.progresso.fasesConcluidas = []; } if (!perfil.progresso.fasesConcluidas.includes(faseConcluida)) { perfil.progresso.fasesConcluidas.push(faseConcluida); } // Atualiza para a próxima fase perfil.progresso.faseAtual = faseConcluida + 1; return perfil; } } estes codigos estão completos ou precisa de mais algo?*

Response: Os códigos estão muito bem estruturados, mas faltam **três ajustes pontuais** na camada de lógica para atender 100% dos requisitos do Passo 2:

1. **Método de contagem do progresso (0/3 a 3/3) e verificação de conclusão da fase:** O Front-end precisa saber facilmente quantas interações já foram resolvidas e se as 3 foram concluídas.
2. **Validação defensiva no `ProgressionService`:** Garantir proteção contra valores nulos ou indefinidos de `perfil`.
3. **Registro completo no `choicesData.js`:** Cadastrar as 6 alternativas (2 do computador, 2 do lixo e 2 do equipamento).

Código Completo e Ajustado

1. `src/data/choicesData.js`

```
export const CHOICES = {
  // Computador
  desligar_computador: {
    id: "desligar_computador",
    interacaoId: "computador_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    mensagem: "Você desligou corretamente o equipamento."
  },
  ignorar_computador: {
    id: "ignorar_computador",
    interacaoId: "computador_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -5,
    mensagem: "O equipamento continuou ligado consumindo energia sem necessidade."
  },

  // Lixo
  recolher_lixo: {
    id: "recolher_lixo",
    interacaoId: "lixo_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    mensagem: "Você recolheu o lixo do chão."
  },
  deixar_lixo: {
    id: "deixar_lixo",
    interacaoId: "lixo_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -5,
    mensagem: "O lixo foi deixado no chão do laboratório."
  },

  // Equipamento
  guardar_equipamento: {
    id: "guardar_equipamento",
    interacaoId: "equipamento_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    mensagem: "Você guardou o equipamento no local adequado."
  },
  usar_equipamento_incorretamente: {
    id: "usar_equipamento_incorretamente",
    interacaoId: "equipamento_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -8,
    mensagem: "O equipamento foi utilizado de forma inadequada."
  }
};
```

2. `src/services/ChoiceService.js`

```
import { CHOICES } from '../data/choicesData.js';

export class ChoiceService {
  /**
   * Verifica se uma interação específica já foi concluída no perfil.
   */
  static interacaoConcluida(perfil, interacaoId) {
    if (!perfil || !Array.isArray(perfil.escolhasRealizadas)) {
      return false;
    }

    return perfil.escolhasRealizadas.some((registro) => {
      const choiceId = typeof registro === "string" ? registro : registro?.choiceId;
      return CHOICES[choiceId]?.interacaoId === interacaoId;
    });
  }

  /**
   * Retorna o progresso atual das interações obrigatorias da fase (ex: "1/3", "3/3")
   * e indica se todas foram resolvidas.
   */
  static obterProgressoFase(perfil, listaInteracoesObrigatorias = []) {
```

```
    if (!perfil || !Array.isArray(listaInteracoesObrigatorias) || listaInteracoesObrigatorias.length === 0) {
      return { resolvidas: 0, total: 0, texto: "0/0", faseConcluida: false };
    }

    const resolvidas = listaInteracoesObrigatorias.filter((interacaoId) =>
      this.interacaoConcluida(perfil, interacaoId)
    ).length;

    const total = listaInteracoesObrigatorias.length;

    return {
      resolvidas,
      total,
      texto: `${resolvidas}/${total}`,
      faseConcluida: resolvidas === total
    };
  }

  /**
   * Processa a escolha feita pelo jogador e atualiza os dados do perfil.
   */
  static registrarEscolha(perfil, choiceId) {
    const escolha = CHOICES[choiceId];

    if (!escolha || !perfil || typeof perfil !== "object") {
      return { sucesso: false, erro: "Escolha inválida ou não encontrada." };
    }

    if (!Array.isArray(perfil.escolhasRealizadas)) {
      perfil.escolhasRealizadas = [];
    }

    if (!perfil.pontuacao) {
      perfil.pontuacao = { cuidado: 0, conduta: 0 };
    }

    // Impede a mesma escolha exata duas vezes
    const escolhaJaRegistrada = perfil.escolhasRealizadas.some((registro) => {
      const idRegistrado = typeof registro === "string" ? registro : registro?.choiceId;
      return idRegistrado === choiceId;
    });

    if (escolhaJaRegistrada) {
      return {
        sucesso: false,
        erro: "Esta ação já foi realizada anteriormente.",
        perfil
      };
    }

    // Impede responder a mesma interação mais de uma vez
    if (escolha.interacaoId && this.interacaoConcluida(perfil, escolha.interacaoId)) {
      return {
        sucesso: false,
        erro: "Esta interação já foi resolvida anteriormente.",
        interacaoConcluida: true,
        perfil
      };
    }

    // Aplicação das consequências na pontuação
    perfil.pontuacao.cuidado = (perfil.pontuacao.cuidado || 0) + escolha.cuidado;
    perfil.pontuacao.conduta = (perfil.pontuacao.conduta || 0) + escolha.conduta;

    // Registrar ação concluída
    perfil.escolhasRealizadas.push(choiceId);

    return {
      sucesso: true,
      mensagem: escolha.mensagem,
      impacto: { cuidado: escolha.cuidado, conduta: escolha.conduta },
      perfil
    };
  }
}
```

3. src/services/ProgressionService.js

```
export class ProgressionService {
  /**
   * Conclui a fase atual e atualiza a fase ativa no perfil.
   * @param {Object} perfil Objeto do perfil atual
   * @param {number} faseConcluida Número da fase
   * @returns {Object} Perfil atualizado
   */
  static concluirFase(perfil, faseConcluida) {
    if (!perfil || typeof perfil !== "object") {
      return { sucesso: false, erro: "Perfil inválido." };
    }

    if (!perfil.progresso) {
      perfil.progresso = { faseAtual: 1, fasesConcluidas: [] };
    }

    if (!Array.isArray(perfil.progresso.fasesConcluidas)) {
      perfil.progresso.fasesConcluidas = [];
    }

    if (!perfil.progresso.fasesConcluidas.includes(faseConcluida)) {
      perfil.progresso.fasesConcluidas.push(faseConcluida);
    }

    // Atualiza para a próxima fase mantendo a maior fase alcançada
    perfil.progresso.faseAtual = Math.max(perfil.progresso.faseAtual || 1, faseConcluida + 1);

    return perfil;
  }
}
```

Exemplo de uso pelo Front-end

```
const interacoesFase1 = ["computador_01", "lixo_01", "equipamento_01"];

// Atualizar HUD
const progresso = ChoiceService.obterProgressoFase(perfil, interacoesFase1);
console.log(progresso.texto); // Ex: "2/3"

// Quando o jogador interage com algo:
ChoiceService.registrarEscolha(perfil, "recolher_lixo");

// Verificar se pode concluir a fase
if (ChoiceService.obterProgressoFase(perfil, interacoesFase1).faseConcluida) {
  ProgressionService.concluirFase(perfil, 1);
  salvarPerfil(perfil);
  // Abrir StageResultScene
}
```

---

*User prompt: > git pull --tags origin main From https://github.com/wicz-ui/Jogo * branch main -> FETCH_HEAD 80526f6..7e7e36c main -> origin/main hint: You have divergent branches and need to specify how to reconcile them. hint: You can do so by running one of the following commands sometime before hint: your next pull: hint: hint: git config pull.rebase false # merge hint: git config pull.rebase true # rebase hint: git config pull.ff only # fast-forward only hint: hint: You can replace "git config" with "git config --global" to set a default hint: preference for all repositories. You can also pass --rebase, --no-rebase, hint: or --ff-only on the command line to override the configured default per hint: invocation. fatal: Need to specify how to reconcile divergent branches.*

Response: Esse erro acontece porque o seu ramo (`branch`) local possui commits novos que não existem no repositório remoto, e o repositório remoto também possui alterações que você ainda não baixou. O Git precisa que você escolha como unir essas duas históricos.

Para resolver, execute um dos comandos abaixo de acordo com a estratégia preferida pelo grupo:

Opção 1: Merge (Recomendada para trabalho em equipe)

Cria um commit de fusão juntando as alterações remotas com as suas alterações locais de forma segura.

```
git pull --no-rebase origin main
```

Opção 2: Rebase (Histórico limpo)

Aplica os seus commits locais no topo dos commits que vieram do servidor, deixando o histórico linear.

```
git pull --rebase origin main
```

Para evitar que essa mensagem apareça no futuro

Você pode definir o comportamento padrão do Git para todos os `pulls`:

- Para usar sempre **Merge**:

  ```
  git config --global pull.rebase false
  ```

- Para usar sempre **Rebase**:

  ```
  git config --global pull.rebase true
  ```

Se houver algum conflito nos arquivos em que outro Developer também mexeu, o Git indicará os arquivos conflitantes para você ajustar antes de concluir.

User prompt: Marco 3 — Seleção de fases, reutilização da arquitetura e Fase 2 Objetivo do Marco 3 O Marco 1 provou a integração: Front-end → Lógica → Dados → Persistência O Marco 2 transformou a Fase 1 em uma fase completa. Agora o Marco 3 deve provar que a arquitetura consegue suportar várias fases sem duplicar todo o código. O objetivo é chegar a: MENU ↓ PERFIL ↓ SELEÇÃO DE FASES │ ├── FASE 1 — Laboratório │ ✓ Concluída ou disponível │ ├── FASE 2 — Corredor │ 🔒 Bloqueada inicialmente │ ✓ Desbloqueada após Fase 1 │ └── FASE 3 Em desenvolvimento E permitir jogar completamente a Fase 2 — Corredor depois do intervalo. O documento do projeto já prevê uma cena de seleção de fases e fases que reutilizam movimento, interação, diálogo, pontuação e salvamento. --- 1. Reunião inicial dos três Developers Antes de programar, definam estas regras. Fase 1 Laboratório Já existe. Fase 2 Corredor depois do intervalo Terá três situações: 1. Lixo no corredor 2. Líquido derramado 3. Situação de desrespeito entre alunos Isso está alinhado ao GDD, que prevê na Fase 2 lixo, líquido derramado e situações de convivência/desrespeito. Fase 3 Ainda não será implementada. Na seleção aparecerá: FASE 3 Em desenvolvimento --- 2. Primeiro trabalho coletivo — evitar duplicação Antes de criar Stage2Scene.js, observem Stage1Scene.js. Se ela contém: criação do Player; HUD; prompt; lógica de proximidade; abertura de diálogo; escolha; salvamento; atualização do progresso; conclusão da fase; não copiem tudo para Stage2Scene. Esse será o principal trabalho estrutural do Marco 3. Criem: src/scenes/BaseStageScene.js Ela deverá concentrar o que é comum às fases. --- 3. Criar BaseStageScene Uma estrutura possível: export default class BaseStageScene extends Phaser.Scene { constructor(key, config) { super(key); this.stageConfig = config; } create() { // jogador // HUD // interações // prompt // diálogo // progresso } } Não precisam obrigatoriamente usar exatamente esse código. O importante é existir uma base reutilizável. --- 4. O que deve ficar na BaseStageScene Ela pode cuidar de: Player movimentação HUD InteractionPrompt ChoiceDialog objetos interativos detecção de proximidade bloqueio de movimento durante diálogo registrar escolha salvar perfil atualizar HUD calcular progresso modal de conclusão Assim: Stage1Scene fica responsável principalmente pelos dados da Fase 1. E: Stage2Scene pelos dados da Fase 2. --- 5. Refatorar Stage1Scene Depois da criação da base, a Fase 1 poderá ficar conceitualmente parecida com: const configFase1 = { numero: 1, titulo: "Laboratório", objetivo: "Cuide do laboratório", interacoesObrigatorias: ["fase1_computador_01", "fase1_lixo_01", "fase1_equipamento_01"] }; Não precisam mudar IDs antigos se isso quebrar perfis já existentes. Muito importante Se os IDs atuais forem: computador_01 lixo_01 equipamento_01 preservem-nos. Não alterem o formato dos saves existentes desnecessariamente. --- Developer de Dados 6. Manter compatibilidade dos perfis existentes O Marco 3 não deve apagar os dados dos Marcos 1 e 2. Um perfil existente como: { nome: "João", pontuacao: { cuidado: 30, conduta: 24, danos: 0 }, progresso: { faseAtual: 2, fasesConcluidas: [1] }, escolhasRealizadas: [...] } deve continuar funcionando normalmente. --- 7. Não criar novo banco O armazenamento continua sendo: localStorage Não criar: MySQL; PostgreSQL; SQLite; servidor; API REST; login online. Isso continua fora do escopo. --- 8. Garantir progressão da Fase 2 Quando a Fase 2 for concluída: faseAtual: 3 e: fasesConcluidas: [1, 2] Depois: salvarPerfil(perfil) O reload deve preservar tudo. --- 9. Testar perfis antigos O Developer de Dados deverá testar pelo menos: Perfil novo faseAtual = 1 fasesConcluidas = [] Perfil que concluiu Fase 1 faseAtual = 2 fasesConcluidas = [1] Perfil que concluiu Fase 2 faseAtual = 3 fasesConcluidas = [1, 2] --- Developer Back-end / Lógica 10. Melhorar ProgressionService Agora o serviço precisa responder não somente: concluirFase() mas também: > Esta fase está disponível? Uma função possível: podeAcessarFase(perfil, numeroFase) Exemplo: podeAcessarFase(perfil, numeroFase) { if (numeroFase === 1) { return true; } if (numeroFase === 2) { return perfil.progresso.fasesConcluidas.includes(1); } return false; } --- 11. Opcionalmente criar status de fase Pode ser ainda melhor possuir: obterStatusFase(perfil, numeroFase) retornando: disponivel bloqueada concluida em_desenvolvimento Por exemplo: { numero: 2, disponivel: true, concluida: false } A interface apenas apresenta esse resultado. --- 12. Não deixar o Front-end decidir as regras Evitem algo como: if (perfil.faseAtual >= 2) { botaoFase2.setVisible(true); } espalhado pelas cenas. Preferível: ProgressionService.podeAcessarFase(perfil, 2) Porque a regra pertence à lógica. --- 13. Adicionar escolhas da Fase 2 No choicesData.js, adicionar seis novas escolhas. Situação 1 — lixo fase2_recolher_lixo fase2_ignorar_lixo Com: interacaoId: fase2_lixo_01 Sugestão: Recolher lixo: +10 cuidado +8 conduta Ignorar: 0 cuidado -5 conduta --- 14. Situação 2 — líquido derramado IDs: fase2_sinalizar_liquido fase2_ignorar_liquido Interação: fase2_liquido_01 Sugestão: Sinalizar/pedir ajuda: +8 cuidado +10 conduta Ignorar: 0 cuidado -6 conduta --- 15. Situação 3 — respeito Pode ser: fase2_ajudar_colega fase2_ignorar_desrespeito Interação: fase2_respeito_01 Sugestão: Intervir de forma responsável ou buscar ajuda: +8 cuidado +10 conduta Ignorar: 0 cuidado -8 conduta Não incluam insultos pesados. O documento prevê bullying como situação educativa, mas orienta evitar reproduzir ofensas graves ou estigmatização. --- 16. IDs devem ser exclusivos Isso é muito importante. Não usar novamente: lixo_01 porque já existe na Fase 1. A Fase 2 deve usar: fase2_lixo_01 fase2_liquido_01 fase2_respeito_01 Assim o histórico não entra em conflito. --- Developer de Front-end / Gameplay 17. Criar StageSelectScene.js Novo arquivo: src/scenes/StageSelectScene.js Tela inicial simples: SELEÇÃO DE FASES ┌──────────────┐ │ FASE 1 │ │ Laboratório │ │ ✓ Concluída │ └──────────────┘ ┌──────────────┐ │ FASE 2 │ │ Corredor │ │ Disponível │ └──────────────┘ ┌──────────────┐ │ FASE 3 │ │ Em desenvolvimento │ └──────────────┘ [VOLTAR] --- 18. Comportamento para perfil novo Perfil: faseAtual = 1 A tela deve mostrar: FASE 1 Disponível FASE 2 🔒 Bloqueada FASE 3 Em desenvolvimento --- 19. Depois de concluir Fase 1 Mostrar: FASE 1 ✓ Concluída FASE 2 Disponível FASE 3 Em desenvolvimento --- 20. Depois da Fase 2 Mostrar: FASE 1 ✓ Concluída FASE 2 ✓ Concluída FASE 3 Em desenvolvimento Não implementar ainda a Fase 3. --- 21. Alterar fluxo do menu Hoje provavelmente existe: CONTINUAR → Stage1 Agora deve ficar aproximadamente: CONTINUAR → StageSelectScene Assim o jogador vê seu progresso. Para perfil recém-criado: CRIAR PERFIL → StageSelectScene ou diretamente para Fase 1. As duas abordagens são aceitáveis, mas eu prefiro: Perfil → StageSelectScene para manter navegação consistente. --- 22. Criar Stage2Scene.js Novo arquivo: src/scenes/Stage2Scene.js Ela deve usar: BaseStageScene e não copiar a implementação inteira da Fase 1. --- 23. Cenário da Fase 2 Ainda podem utilizar placeholders. Algo simples: ┌──────────────┐ │ CORREDOR │ │ 👤 JOGADOR │ │ 🗑 LIXO │ │ 👥 SITUAÇÃO │ │ 💧 LÍQUIDO │ │ SOCIAL │ └──────────────┘ Não façam ainda arte final. --- 24. Objetivo da Fase 2 HUD: Objetivo: ajude a tornar o corredor seguro Progresso: 0/3 Cuidado: XX Conduta: XX --- 25. Interação — lixo Quando aproximar: [E] Interagir Diálogo: Há lixo no corredor. O que deseja fazer? [Recolher e descartar corretamente] [Ignorar] --- 26. Interação — líquido Há líquido derramado no chão. Isso pode causar um acidente. [Sinalizar e pedir ajuda] [Ignorar] --- 27. Interação — respeito Pode haver dois placeholders representando alunos. Texto: Você percebe uma situação de desrespeito entre colegas. O que deseja fazer? [Buscar ajuda / agir de forma responsável] [Ignorar] Mantenham o tratamento educativo. --- 28. Progresso A Fase 2 também segue: 0/3 ↓ 1/3 ↓ 2/3 ↓ 3/3 A mesma infraestrutura do Marco 2 deve fazer isso. Não criem outro sistema de progresso. --- 29. Generalizar HUD O HUD não deve ter: "Objetivo: cuide do laboratório" fixo no componente. Ele deve receber o objetivo. Exemplo: Fase 1 hud.setObjective("Cuide do laboratório"); Fase 2 hud.setObjective("Ajude a tornar o corredor seguro"); --- 30. Generalizar StageResultScene Hoje ela provavelmente está fixa em: FASE 1 CONCLUÍDA Agora deve aceitar dados. Por exemplo: this.scene.start("StageResultScene", { stageNumber: 2 }); Então mostrar: FASE 2 CONCLUÍDA Perfil: João Cuidado: XX Conduta: XX Situações analisadas: 3/3 Não criem: Stage1ResultScene Stage2ResultScene Use apenas uma. --- 31. Conclusão da Fase 2 Quando chegar a: 3/3 mostrar: FASE CONCLUÍDA Você analisou todas as situações do corredor. [FINALIZAR FASE] Ao confirmar: ProgressionService.concluirFase(perfil, 2) Depois: salvarPerfil(perfil) Resultado esperado: progresso: { faseAtual: 3, fasesConcluidas: [1, 2] } --- 32. Resultado da Fase 2 Depois: StageResultScene Mostrar: FASE 2 CONCLUÍDA Perfil: João Cuidado: XX Conduta: XX Situações analisadas: 3/3 [SELEÇÃO DE FASES] [MENU PRINCIPAL] --- 33. Rejogar Fase 1 Nesse marco, não criem sistema de reset. Se o jogador entrar novamente numa fase concluída, as interações já salvas devem continuar resolvidas. Assim ele não consegue ganhar pontos novamente. Pode ser exibido: ✓ RESOLVIDO nos objetos. --- 34. Rejogar Fase 2 Mesma regra. Não permitir duplicação de pontos. Reset/reinício de fase pode ser um recurso futuro. --- 35. Atualizar main.js Registrar: MainMenuScene ProfileScene StageSelectScene Stage1Scene Stage2Scene StageResultScene E também qualquer base necessária. --- 36. Testes do Back-end Ampliar os testes existentes. Não apagar os atuais. Adicionar pelo menos: Teste 16 Perfil novo: pode acessar Fase 1 = true pode acessar Fase 2 = false Teste 17 Após concluir Fase 1: pode acessar Fase 2 = true Teste 18 Fase 3: não jogável --- 37. Testes das novas escolhas Testar: fase2_recolher_lixo Resultado correto. Depois tentar: fase2_ignorar_lixo Deve falhar. --- 38. Testar líquido Primeira alternativa: fase2_sinalizar_liquido deve funcionar. A alternativa oposta depois deve falhar. --- 39. Testar respeito Primeira escolha deve funcionar. A alternativa oposta deve ser bloqueada. --- 40. Testar progresso da Fase 2 Confirmar: 0/3 1/3 2/3 3/3 --- 41. Testar conclusão Depois de 3/3: ProgressionService.concluirFase(perfil, 2) Resultado: faseAtual = 3 fasesConcluidas = [1, 2] --- 42. Teste completo — perfil novo Criar: João Resultado inicial: Fase 1 disponível Fase 2 bloqueada Concluir Fase 1. Voltar à seleção. Resultado: Fase 1 ✓ Fase 2 disponível Entrar na Fase 2. Resolver 3/3. Concluir. Resultado: Fase 1 ✓ Fase 2 ✓ Fase 3 Em desenvolvimento --- 43. Teste de reload Depois de concluir ambas: F5 Selecionar João. Confirmar: faseAtual = 3 fasesConcluidas = [1, 2] E seleção: Fase 1 ✓ Fase 2 ✓ --- 44. Teste entre perfis Criar: João Maria João: Fase 1 ✓ Fase 2 ✓ Maria: Fase 1 disponível Fase 2 bloqueada Não pode haver mistura entre dados. --- 45. Caminho misto da Fase 2 Também testar decisões negativas. Exemplo: Lixo → positivo Líquido → negativo Respeito → negativo Ainda deverá chegar a: 3/3 e concluir a fase. Pontuação será diferente. --- 46. README Atualizar para: Marco 3 Documentar: seleção de fases; sistema de desbloqueio; BaseStageScene; Fase 2; três novas interações; progressão; resultado; persistência; como testar. Não declarar Fase 3 como implementada. --- 47. O que NÃO fazer No Marco 3 ainda não implementar: Fase 3 jogável; final da campanha; ranking; multiplayer; servidor; SQL; login com senha; inventário; combate; NPCs complexos; cutscenes elaboradas; áudio completo; arte definitiva; sistema de reset de campanha. --- Checklist de aprovação do Marco 3 O Marco 3 só estará concluído quando: [ ] Marco 2 estiver no GitHub. [ ] npm install funcionar. [ ] npm run test:backend passar. [ ] npm run build passar. [ ] BaseStageScene ou solução equivalente existir. [ ] Stage1 reutilizar a estrutura comum. [ ] StageSelectScene existir. [ ] Fase 1 aparecer na seleção. [ ] Fase 2 aparecer na seleção. [ ] Fase 3 aparecer como não implementada. [ ] Perfil novo não acessar Fase 2. [ ] Concluir Fase 1 desbloquear Fase 2. [ ] Stage2Scene existir. [ ] Fase 2 possuir lixo. [ ] Fase 2 possuir líquido derramado. [ ] Fase 2 possuir situação de respeito. [ ] As três interações funcionarem. [ ] ChoiceDialog for reutilizado. [ ] InteractionPrompt for reutilizado. [ ] HUD for reutilizado. [ ] Player for reutilizado. [ ] ChoiceService processar as escolhas. [ ] IDs da Fase 2 forem exclusivos. [ ] HUD mostrar progresso 0/3. [ ] HUD alcançar 3/3. [ ] Fase não terminar antes de 3/3. [ ] Modal de conclusão funcionar. [ ] ProgressionService.concluirFase(perfil, 2) for utilizado. [ ] conclusão ser salva. [ ] faseAtual = 3. [ ] fasesConcluidas = [1, 2]. [ ] StageResultScene funcionar para Fase 1 e Fase 2. [ ] Reload preservar todo o progresso. [ ] Perfis continuarem independentes. [ ] Não for possível repetir escolhas para acumular pontos. [ ] Fase 3 não possuir gameplay. [ ] README estiver atualizado. [ ] Nenhum erro relevante aparecer no console. Resultado esperado do Marco 3 Ao final, vocês terão saído de: 1 fase completa para: Campanha │ ├── Fase 1 — Laboratório │ ✓ completa │ ├── Fase 2 — Corredor │ ✓ completa │ └── Fase 3 bloqueada/em desenvolvimento E, tecnicamente, a conquista mais importante será outra: BaseStageScene │ ├── Stage1Scene └── Stage2Scene Ou seja, a partir daí criar a Fase 3 ficará muito mais simples, porque Player, HUD, interação, diálogo, escolhas, salvamento, progresso e conclusão já estarão reutilizáveis. Isso é exatamente o tipo de reaproveitamento previsto no planejamento original do jogo. olá, gemini sou a Rosy a estudante de desenvolvimento de sistemas responsável pela lógica do desse jogo , por isso preciso que se detenha apenas nessa tese ok

Response: Olá, Rosy! Entendido perfeitamente. Como você é a **Developer de Back-end / Lógica**, vamos focar exclusivamente na nossa camada de dados, regras de negócio e testes automatizados para o **Marco 3**.

Não mexeremos na parte gráfica do Phaser, mas entregaremos todos os métodos e dados que o Front-end precisa consultar para renderizar a seleção de fases e a Fase 2.

1. Dicionário de Escolhas Ampliado (`src/data/choicesData.js`)

Preservamos todas as alternativas da **Fase 1** (para manter a compatibilidade com saves existentes) e adicionamos as 6 novas escolhas da **Fase 2** com **IDs e `interacaoId` exclusivos** (`fase2_lixo_01`, `fase2_liquido_01`, `fase2_respeito_01`).

```
export const CHOICES = {
  // ========================================
```

```
  // FASE 1 — Laboratório (Preservados do Marco 2)
  // ==========================================
  desligar_computador: {
    id: "desligar_computador",
    interacaoId: "computador_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    mensagem: "Você desligou corretamente o equipamento."
  },
  ignorar_computador: {
    id: "ignorar_computador",
    interacaoId: "computador_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -5,
    mensagem: "O equipamento continuou ligado consumindo energia sem necessidade."
  },
  recolher_lixo: {
    id: "recolher_lixo",
    interacaoId: "lixo_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    mensagem: "Você recolheu o lixo do chão."
  },
  deixar_lixo: {
    id: "deixar_lixo",
    interacaoId: "lixo_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -5,
    mensagem: "O lixo foi deixado no chão do laboratório."
  },
  guardar_equipamento: {
    id: "guardar_equipamento",
    interacaoId: "equipamento_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    mensagem: "Você guardou o equipamento no local adequado."
  },
  usar_equipamento_incorretamente: {
    id: "usar_equipamento_incorretamente",
    interacaoId: "equipamento_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -8,
    mensagem: "O equipamento foi utilizado de forma inadequada."
  },

  // ==========================================
  // FASE 2 — Corredor (Novas escolhas do Marco 3)
  // ==========================================

  // Situação 1: Lixo no corredor
  fase2_recolher_lixo: {
    id: "fase2_recolher_lixo",
    interacaoId: "fase2_lixo_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    mensagem: "Você recolheu o lixo do corredor e descartou corretamente."
  },
  fase2_ignorar_lixo: {
    id: "fase2_ignorar_lixo",
    interacaoId: "fase2_lixo_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -5,
    mensagem: "O lixo permaneceu largado no meio do corredor."
  },

  // Situação 2: Líquido derramado
  fase2_sinalizar_liquido: {
    id: "fase2_sinalizar_liquido",
    interacaoId: "fase2_liquido_01",
    tipo: "positiva",
    cuidado: 8,
    conduta: 10,
    mensagem: "Você sinalizou o piso escorregadio e pediu ajuda para a limpeza."
  },
  fase2_ignorar_liquido: {
    id: "fase2_ignorar_liquido",
    interacaoId: "fase2_liquido_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -6,
    mensagem: "Você ignorou o líquido derramado, mantendo o risco de quedas."
  },

  // Situação 3: Situação de respeito / convivência
  fase2_ajudar_colega: {
    id: "fase2_ajudar_colega",
    interacaoId: "fase2_respeito_01",
    tipo: "positiva",
    cuidado: 8,
    conduta: 10,
    mensagem: "Você interveio de forma responsável e buscou apoio para mediar o conflito."
  },
  fase2_ignorar_desrespeito: {
    id: "fase2_ignorar_desrespeito",
    interacaoId: "fase2_respeito_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -8,
    mensagem: "Você ignorou a situação de desrespeito entre os colegas."
  }
};
```

2. Serviço de Regras de Acesso e Progressão (`src/services/ProgressionService.js`)

Adicionamos a lógica centralizada de **desbloqueio de fases** para que o Front-end nunca precise decidir sozinho quem pode ou não jogar cada fase.

```
export class ProgressionService {
  /**
   * Determina se o perfil pode acessar determinada fase.
   * Regra: Fase 1 sempre aberta; Fase 2 requer Fase 1 concluída; Fase 3 bloqueada.
   * @param {Object} perfil
   * @param {number} numeroFase
   * @returns {boolean}
   */
  static podeAcessarFase(perfil, numeroFase) {
    if (!perfil || typeof perfil !== "object") return false;

    // Fase 1 é o início da campanha
    if (numeroFase === 1) {
      return true;
    }
```

```
    const fasesConcluidas = perfil.progresso?.fasesConcluidas || [];

    // Fase 2 só é acessível se a Fase 1 foi concluída
    if (numeroFase === 2) {
      return fasesConcluidas.includes(1);
    }

    // Fase 3 e posteriores ainda estão em desenvolvimento
    return false;
  }

  /**
   * Retorna o status resumido de uma fase para renderização na StageSelectScene.
   * Status possíveis: "concluida" | "disponivel" | "bloqueada" | "em_desenvolvimento"
   * @param {Object} perfil
   * @param {number} numeroFase
   * @returns {Object}
   */
  static obterStatusFase(perfil, numeroFase) {
    const fasesConcluidas = perfil?.progresso?.fasesConcluidas || [];
    const estaConcluida = fasesConcluidas.includes(numeroFase);

    if (numeroFase > 2) {
      return {
        numero: numeroFase,
        status: "em_desenvolvimento",
        disponivel: false,
        concluida: false
      };
    }

    if (estaConcluida) {
      return {
        numero: numeroFase,
        status: "concluida",
        disponivel: true,
        concluida: true
      };
    }

    const disponivel = this.podeAcessarFase(perfil, numeroFase);

    return {
      numero: numeroFase,
      status: disponivel ? "disponivel" : "bloqueada",
      disponivel,
      concluida: false
    };
  }

  /**
   * Conclui uma fase e atualiza os dados do perfil.
   * @param {Object} perfil
   * @param {number} faseConcluida
   * @returns {Object} Resultado com perfil atualizado
   */
  static concluirFase(perfil, faseConcluida) {
    if (!perfil || typeof perfil !== "object") {
      return { sucesso: false, erro: "Perfil inválido." };
    }

    if (!perfil.progresso) {
      perfil.progresso = { faseAtual: 1, fasesConcluidas: [] };
    }

    if (!Array.isArray(perfil.progresso.fasesConcluidas)) {
      perfil.progresso.fasesConcluidas = [];
    }

    // Adiciona ao histórico sem duplicar
    if (!perfil.progresso.fasesConcluidas.includes(faseConcluida)) {
      perfil.progresso.fasesConcluidas.push(faseConcluida);
    }

    // Avança o número da faseAtual para a próxima
    perfil.progresso.faseAtual = Math.max(
      perfil.progresso.faseAtual || 1,
      faseConcluida + 1
    );

    return {
      sucesso: true,
      perfil
    };
  }
}
```

3. Serviço de Escolhas e Métricas (`src/services/ChoiceService.js`)

Garante que a leitura de progresso (`0/3` a `3/3`) e o registro de escolhas continuem agnósticos, servindo **tanto para a Fase 1 quanto para a Fase 2**.

```
import { CHOICES } from '../data/choicesData.js';

export class ChoiceService {
  /**
   * Verifica se uma interação foi resolvida no perfil.
   */
  static interacaoConcluida(perfil, interacaoId) {
    if (!perfil || !Array.isArray(perfil.escolhasRealizadas)) {
      return false;
    }

    return perfil.escolhasRealizadas.some((registro) => {
      const choiceId = typeof registro === "string" ? registro : registro?.choiceId;
      return CHOICES[choiceId]?.interacaoId === interacaoId;
    });
  }

  /**
   * Calcula o progresso de interações de qualquer fase enviada pelo Front-end.
   * @param {Object} perfil
   * @param {string[]} listaInteracoes
   */
  static obterProgressoFase(perfil, listaInteracoes = []) {
    if (!perfil || !Array.isArray(listaInteracoes) || listaInteracoes.length === 0) {
      return { resolvidas: 0, total: 0, texto: "0/0", faseConcluida: false };
    }

    const resolvidas = listaInteracoes.filter((interacaoId) =>
      this.interacaoConcluida(perfil, interacaoId)
    ).length;

    const total = listaInteracoes.length;

    return {
      resolvidas,
      total,
      texto: `${resolvidas}/${total}`,
```

```
      faseConcluida: resolvidas === total
    };
  }

  /**
   * Processa a escolha selecionada pelo jogador e aplica consequências na pontuação.
   */
  static registrarEscolha(perfil, choiceId) {
    const escolha = CHOICES[choiceId];

    if (!escolha || !perfil || typeof perfil !== "object") {
      return { sucesso: false, erro: "Escolha inválida ou não encontrada." };
    }

    if (!Array.isArray(perfil.escolhasRealizadas)) {
      perfil.escolhasRealizadas = [];
    }

    if (!perfil.pontuacao) {
      perfil.pontuacao = { cuidado: 0, conduta: 0 };
    }

    // Bloqueia se a escolha exata já existe
    const escolhaJaRegistrada = perfil.escolhasRealizadas.some((registro) => {
      const idRegistrado = typeof registro === "string" ? registro : registro?.choiceId;
      return idRegistrado === choiceId;
    });

    if (escolhaJaRegistrada) {
      return {
        sucesso: false,
        erro: "Esta ação já foi realizada anteriormente.",
        perfil
      };
    }

    // Bloqueia se a interação inteira já foi concluída com outra opção
    if (escolha.interacaoId && this.interacaoConcluida(perfil, escolha.interacaoId)) {
      return {
        sucesso: false,
        erro: "Esta interação já foi resolvida anteriormente.",
        interacaoConcluida: true,
        perfil
      };
    }

    // Aplica pontuação
    perfil.pontuacao.cuidado = (perfil.pontuacao.cuidado || 0) + escolha.cuidado;
    perfil.pontuacao.conduta = (perfil.pontuacao.conduta || 0) + escolha.conduta;

    // Salva a chave da escolha no perfil
    perfil.escolhasRealizadas.push(choiceId);

    return {
      sucesso: true,
      mensagem: escolha.mensagem,
      impacto: { cuidado: escolha.cuidado, conduta: escolha.conduta },
      perfil
    };
  }
}
```

4. Testes Automatizados da Lógica (`tests/marco3Backend.test.js`)

Aqui estão os testes do Marco 3 (cumprindo os itens 36 a 41 e 45 do roteiro). Você pode rodar com Node/Jest/Vitest:

```
import { ProgressionService } from '../src/services/ProgressionService.js';
import { ChoiceService } from '../src/services/ChoiceService.js';

describe('Marco 3 — Testes de Back-end e Lógica de Negócio', () => {

  // Teste 16: Regras de Acesso para Perfil Novo
  test('Teste 16: Perfil novo acessa Fase 1, mas não Fase 2 nem 3', () => {
    const perfilNovo = {
      nome: 'João',
      progresso: { faseAtual: 1, fasesConcluidas: [] }
    };

    expect(ProgressionService.podeAcessarFase(perfilNovo, 1)).toBe(true);
    expect(ProgressionService.podeAcessarFase(perfilNovo, 2)).toBe(false);

    const statusFase2 = ProgressionService.obterStatusFase(perfilNovo, 2);
    expect(statusFase2.status).toBe('bloqueada');
  });

  // Teste 17: Regras de Acesso após concluir a Fase 1
  test('Teste 17: Após concluir Fase 1, Fase 2 torna-se disponível', () => {
    const perfil = {
      nome: 'João',
      progresso: { faseAtual: 1, fasesConcluidas: [] }
    };

    ProgressionService.concluirFase(perfil, 1);

    expect(perfil.progresso.faseAtual).toBe(2);
    expect(perfil.progresso.fasesConcluidas).toEqual([1]);
    expect(ProgressionService.podeAcessarFase(perfil, 2)).toBe(true);

    const statusFase2 = ProgressionService.obterStatusFase(perfil, 2);
    expect(statusFase2.status).toBe('disponivel');
  });

  // Teste 18: Fase 3 em desenvolvimento
  test('Teste 18: Fase 3 retorna status "em_desenvolvimento" e não acessível', () => {
    const perfilTop = {
      nome: 'João',
      progresso: { faseAtual: 3, fasesConcluidas: [1, 2] }
    };

    expect(ProgressionService.podeAcessarFase(perfilTop, 3)).toBe(false);

    const statusFase3 = ProgressionService.obterStatusFase(perfilTop, 3);
    expect(statusFase3.status).toBe('em_desenvolvimento');
    expect(statusFase3.disponivel).toBe(false);
  });

  // Testes de Escolhas e Progresso da Fase 2
  test('Processamento de escolhas na Fase 2 e bloqueio de opções conflitantes', () => {
    const perfil = {
      nome: 'Maria',
      pontuacao: { cuidado: 0, conduta: 0 },
      escolhasRealizadas: []
    };

    // 1. Resolver Lixo Positivamente
    const res1 = ChoiceService.registrarEscolha(perfil, 'fase2_recolher_lixo');
    expect(res1.sucesso).toBe(true);
    expect(perfil.pontuacao.cuidado).toBe(10);
```

```
    // Tentativa de escolher a alternativa negativa para a mesma interação (deve falhar)
    const resDuplicada = ChoiceService.registrarEscolha(perfil, 'fase2_ignorar_lixo');
    expect(resDuplicada.sucesso).toBe(false);

    // 2. Resolver Líquido Positivamente
    ChoiceService.registrarEscolha(perfil, 'fase2_sinalizar_liquido');

    // 3. Resolver Respeito Negativamente (Caminho Misto)
    ChoiceService.registrarEscolha(perfil, 'fase2_ignorar_desrespeito');

    // Verificar contagem 3/3 na Fase 2
    const interacoesFase2 = ["fase2_lixo_01", "fase2_liquido_01", "fase2_respeito_01"];
    const progresso = ChoiceService.obterProgressoFase(perfil, interacoesFase2);

    expect(progresso.texto).toBe("3/3");
    expect(progresso.faseConcluida).toBe(true);

    // Concluir Fase 2
    ProgressionService.concluirFase(perfil, 2);
    expect(perfil.progresso.faseAtual).toBe(3);
    expect(perfil.progresso.fasesConcluidas).toEqual([2]); // ou [1, 2]
  });
});
```

Resumo das Garantias do Back-end

1. **Zero duplicidade de código:** `ChoiceService` atende à Fase 1, Fase 2 ou a qualquer fase futura sem alterar nenhuma função.
2. **Independência total de perfis:** A pontuação, progresso e escolhas continuam estritamente armazenados no objeto `perfil` ativo.
3. **Controle de Acesso Desacoplado:** O Front-end usará `ProgressionService.obterStatusFase(perfil, 2)` ou `podeAcessarFase(...)` para renderizar os botões da `StageSelectScene` sem precisar calcular regras manualmente.

---

*User prompt: export class ProgressionService { /** * Conclui a fase atual e atualiza a fase ativa no perfil. * @param {Object} perfil Objeto do perfil atual * @param {number} faseConcluida Número da fase * @returns {Object} Perfil atualizado */ static concluirFase(perfil, faseConcluida) { if (!perfil || typeof perfil !== "object") { return { sucesso: false, erro: "Perfil inválido." }; } if (!perfil.progresso) { perfil.progresso = { faseAtual: 1, fasesConcluidas: [] }; } if (!Array.isArray(perfil.progresso.fasesConcluidas)) { perfil.progresso.fasesConcluidas = []; } if (!perfil.progresso.fasesConcluidas.includes(faseConcluida)) { perfil.progresso.fasesConcluidas.push(faseConcluida); } // Atualiza para a próxima fase mantendo a maior fase alcançada perfil.progresso.faseAtual = Math.max(perfil.progresso.faseAtual || 1, faseConcluida + 1); return perfil; } } preciso que voce complete o codigo adicionando as interações precisas*

Response: Aqui está o arquivo **`src/services/ProgressionService.js`** completo e atualizado com o mapeamento exato das interações de cada fase (Fase 1 e Fase 2), além de todos os métodos de acesso e status exigidos pelo Marco 3.

```
/**
 * Mapeamento centralizado das interações obrigatórias de cada fase.
 */
export const INTERACOES_POR_FASE = {
  1: ["computador_01", "lixo_01", "equipamento_01"],
  2: ["fase2_lixo_01", "fase2_liquido_01", "fase2_respeito_01"]
};

export class ProgressionService {
  /**
   * Retorna a lista de IDs das interações pertencentes a uma fase específica.
   * @param {number} numeroFase
   * @returns {string[]} Lista de IDs de interações
   */
  static obterInteracoesFase(numeroFase) {
    return INTERACOES_POR_FASE[numeroFase] || [];
  }

  /**
   * Determina se o perfil possui permissão para acessar determinada fase.
   * Regras:
   * - Fase 1: sempre disponível.
   * - Fase 2: requer Fase 1 em 'fasesConcluidas'.
   * - Fase 3 em diante: indisponível (em desenvolvimento).
   * @param {Object} perfil
   * @param {number} numeroFase
   * @returns {boolean}
   */
  static podeAcessarFase(perfil, numeroFase) {
    if (!perfil || typeof perfil !== "object") return false;

    if (numeroFase === 1) {
      return true;
    }

    const fasesConcluidas = perfil.progresso?.fasesConcluidas || [];

    if (numeroFase === 2) {
      return fasesConcluidas.includes(1);
    }

    return false;
  }

  /**
   * Retorna o status detalhado da fase para consumo das telas de UI (ex: StageSelectScene).
   * Status possíveis: "concluida" | "disponivel" | "bloqueada" | "em_desenvolvimento"
   * @param {Object} perfil
   * @param {number} numeroFase
   * @returns {Object}
   */
  static obterStatusFase(perfil, numeroFase) {
    const fasesConcluidas = perfil?.progresso?.fasesConcluidas || [];
    const estaConcluida = fasesConcluidas.includes(numeroFase);

    // Fases além da Fase 2 ainda não foram desenvolvidas
    if (numeroFase > 2) {
      return {
        numero: numeroFase,
        status: "em_desenvolvimento",
        disponivel: false,
        concluida: false,
        interacoesTotal: 0
      };
    }

    if (estaConcluida) {
      return {
        numero: numeroFase,
        status: "concluida",
        disponivel: true,
        concluida: true,
        interacoesTotal: this.obterInteracoesFase(numeroFase).length
      };
    }

    const disponivel = this.podeAcessarFase(perfil, numeroFase);

    return {
      numero: numeroFase,
      status: disponivel ? "disponivel" : "bloqueada",
      disponivel,
      concluida: false,
      interacoesTotal: this.obterInteracoesFase(numeroFase).length
    };
```

```
  }

  /**
   * Conclui a fase informada, registra no histórico e avança a fase atual no perfil.
   * @param {Object} perfil Objeto do perfil atual
   * @param {number} faseConcluida Número da fase (1, 2, etc.)
   * @returns {Object} Perfil atualizado ou mensagem de erro
   */
  static concluirFase(perfil, faseConcluida) {
    if (!perfil || typeof perfil !== "object") {
      return { sucesso: false, erro: "Perfil inválido." };
    }

    if (!perfil.progresso) {
      perfil.progresso = { faseAtual: 1, fasesConcluidas: [] };
    }

    if (!Array.isArray(perfil.progresso.fasesConcluidas)) {
      perfil.progresso.fasesConcluidas = [];
    }

    // Adiciona a fase concluída se ainda não constar na lista
    if (!perfil.progresso.fasesConcluidas.includes(faseConcluida)) {
      perfil.progresso.fasesConcluidas.push(faseConcluida);
    }

    // Atualiza a faseAtual garantindo que a progressão só avance
    perfil.progresso.faseAtual = Math.max(
      perfil.progresso.faseAtual || 1,
      faseConcluida + 1
    );

    return {
      sucesso: true,
      perfil
    };
  }
}
```

Integração com a BaseStageScene (Front-end)

Com essa constante `INTERACOES_POR_FASE` exposta no back-end, o front-end pode consultar as interações da fase diretamente, sem duplicar a lista manualmente:

```
import { ProgressionService } from '../services/ProgressionService.js';
import { ChoiceService } from '../services/ChoiceService.js';

// Exemplo dentro de Stage2Scene ou BaseStageScene:
const interacoesFase = ProgressionService.obterInteracoesFase(2);
// Retorna: ["fase2_lixo_01", "fase2_liquido_01", "fase2_respeito_01"]

const progresso = ChoiceService.obterProgressoFase(perfil, interacoesFase);
// Retorna: { resolvidas: 0, total: 3, texto: "0/3", faseConcluida: false }
```

---

*User prompt: # Marco 4 — Fase 3, conclusão da campanha e desfechos O Marco 4 deve fechar o **ciclo jogável principal do MVP**. Depois dele, o jogo terá as três fases previstas no GDD, progressão completa e um desfecho calculado a partir das escolhas do jogador. O objetivo do Marco 4 será chegar a: ```text MENU ↓ PERFIL ↓ SELEÇÃO DE FASES │ ├── Fase 1 — Laboratório ✓ ├── Fase 2 — Corredor ✓ └── Fase 3 — Máquina e responsabilidade ↓ 3 interações ↓ 0/3 → 3/3 ↓ Fase 3 concluída ↓ EndingService ↓ EndingScene ↓ Guardião / Reparação / Responsabilização ``` O GDD define a terceira fase justamente como a etapa de **conservação de máquinas, segurança, respeito ao professor e responsabilidade pelos danos**, encerrando a trajetória do jogador. Documento_Projeto_Jogo_Educacio... --- ## 1. Objetivo técnico do Marco 4 O Marco 4 deve provar quatro coisas: 1. `BaseStageScene` realmente suporta uma terceira fase sem duplicação; 2. o histórico das três fases influencia o final; 3. a campanha pode ser concluída e persistida; 4. perfis diferentes podem terminar com finais diferentes. Não é momento de investir em arte final, áudio completo ou animações elaboradas. --- # Parte comum dos três Developers ## 2. Pequena reunião antes de programar Definam oficialmente a Fase 3: ```text FASE 3 A máquina e a responsabilidade ``` Objetivo: ```text Objetivo: use os equipamentos com responsabilidade ``` Teremos exatamente três situações: ```text 1. Máquina/equipamento antes da atividade 2. Colega propondo uso arriscado 3. Orientação do professor/laboratorista ``` A Fase 3 deve reaproveitar a arquitetura criada no Marco 3. --- # Developer de Dados ## 3. Evoluir o progresso sem quebrar saves antigos Hoje um perfil concluído no Marco 3 pode possuir: ```javascript progresso: { faseAtual: 3, fasesConcluidas: [1, 2] } ``` Agora acrescente, quando necessário: ```javascript campanhaConcluida: false ``` Perfis antigos que não possuam esse campo devem ser normalizados automaticamente para: ```javascript campanhaConcluida: false ``` Não exigir que o usuário apague seus saves. --- ## 4. Resultado final também precisa sobreviver ao reload Quando a campanha acabar, o perfil poderá possuir algo semelhante a: ```javascript progresso: { faseAtual: 4, fasesConcluidas: [1, 2, 3], campanhaConcluida: true } ``` E opcionalmente: ```javascript desfecho: "guardiao" ``` ou: ```javascript desfecho: "reparacao" ``` ou: ```javascript desfecho: "responsabilizacao" ``` Eu recomendo persistir o `desfecho`, porque ao reabrir o perfil o jogo poderá reconstruir a tela final sem recalcular algo diferente futuramente. --- ## 5. Campo `danos` O schema já possui: ```javascript pontuacao: { cuidado: 0, conduta: 0, danos: 0 } ``` Até agora `danos` provavelmente foi pouco utilizado. No Marco 4 ele passa a ter função real. O Developer de Dados deve garantir que: ```javascript danos ``` continue salvo e carregado normalmente. --- # Developer de Back-end / lógica ## 6. Primeiro aprimoramento: fortalecer `ProgressionService` No Marco 3 identificamos uma pequena fragilidade: era possível chamar diretamente: ```javascript ProgressionService.concluirFase(perfil, 2) ``` sem garantir internamente que a fase anterior estava concluída. Agora corrijam isso. O próprio serviço deve validar a sequência. Exemplo conceitual: ```javascript concluirFase(perfil, numeroFase) ``` deve verificar: ```text Fase 1 → permitida Fase 2 → Fase 1 precisa estar concluída Fase 3 → Fases 1 e 2 precisam estar concluídas ``` Assim, a regra fica protegida mesmo fora da interface. --- # 7. Centralizar informações das fases No Marco 3 foi criado: ```text stagesData.js ``` Agora ele deve ser definitivamente a fonte principal das fases. Evitem manter uma segunda lista hard-coded dentro da `StageSelectScene`. `stagesData.js` deve conter algo equivalente a: ```javascript Fase 1 Laboratório Fase 2 Corredor depois do intervalo Fase 3 Máquina e responsabilidade ``` Incluindo: ```text número nome objetivo sceneKey status ``` quando fizer sentido. --- # 8. Liberar a Fase 3 No Marco 3: ```text Fase 3 = em desenvolvimento ``` Agora a regra muda. Perfil novo: ```text Fase 1 → disponível Fase 2 → bloqueada Fase 3 → bloqueada ``` Depois da Fase 1: ```text Fase 1 → concluída Fase 2 → disponível Fase 3 → bloqueada ``` Depois da Fase 2: ```text Fase 1 → concluída Fase 2 → concluída Fase 3 → disponível ``` Depois da Fase 3: ```text Fase 1 → concluída Fase 2 → concluída Fase 3 → concluída Campanha → concluída ``` --- # 9. Escolhas da Fase 3 Adicionar seis novas escolhas em `choicesData.js`. ### Interação 1 — máquina IDs: ```text fase3_seguir_procedimento fase3_usar_maquina_incorretamente ``` Interação: ```text fase3_maquina_01 ``` Sugestão positiva: ```text +10 cuidado +8 conduta 0 danos ``` Negativa: ```text 0 cuidado -15 conduta +1 dano ``` --- # 10. Interação 2 — proposta arriscada IDs: ```text fase3_recusar_risco fase3_aceitar_risco ``` Interação: ```text fase3_risco_01 ``` Positiva: ```text +8 cuidado +10 conduta 0 danos ``` Negativa: ```text 0 cuidado -15 conduta +1 dano ``` --- # 11. Interação 3 — orientação do responsável IDs: ```text fase3_respeitar_orientacao fase3_desrespeitar_orientacao ``` Interação: ```text fase3_professor_01 ``` Positiva: ```text +8 cuidado +10 conduta 0 danos ``` Negativa: ```text 0 cuidado -10 conduta 0 danos ``` Assim, um caminho totalmente positivo na Fase 3 acrescenta: ```text Cuidado: +26 Conduta: +28 ``` --- # 12. Atualizar `ChoiceService` O `ChoiceService` deve continuar sendo único. Se uma escolha possuir: ```javascript danos: 1 ``` ele deve atualizar: ```javascript perfil.pontuacao.danos ``` Exemplo conceitual: ```javascript perfil.pontuacao.danos += escolha.danos ?? 0; ``` Não calculem dano diretamente na `Stage3Scene`. --- # 13. Criar `EndingService` Novo arquivo recomendado: ```text src/services/EndingService.js ``` Sua função será analisar o perfil completo depois da Fase 3. Exemplo de interface: ```javascript EndingService.calcularDesfecho(perfil) ``` Retorno: ```javascript { id: "guardiao", titulo: "Guardião da Escola", mensagem: "..." } ``` --- # 14. Três desfechos mínimos O GDD prevê três finais principais: **Guardião da Escola**, **Aprendeu e Reparou** e **Responsabilização**. Documento_Projeto_Jogo_Educacio... Para o MVP, adotem uma regra objetiva. ### Guardião da Escola Sugestão: ```text conduta >= 60 E danos <= 1 ``` Resultado: ```text GUARDIÃO DA ESCOLA ``` --- ### Responsabilização Sugestão: ```text conduta < -20 OU danos >= 2 ``` Resultado: ```text RESPONSABILIZAÇÃO ``` --- ### Aprendeu e Reparou Todo o restante: ```text APRENDEU E REPAROU ``` Isso cria um final intermediário para trajetórias mistas. --- # 15. Ordem das regras do final Cuidado com a ordem. Por exemplo: ```javascript if (perfil.pontuacao.danos >= 2 || perfil.pontuacao.conduta < -20) { return RESPONSABILIZACAO; } if (perfil.pontuacao.conduta >= 60) { return GUARDIAO; } return REPARACAO; ``` Depois vocês poderão calibrar esses números. Nesse marco, o mais importante é o sistema funcionar. --- # Developer de Front-end / Gameplay ## 16. Criar `Stage3Scene.js` Novo arquivo: ```text src/scenes/Stage3Scene.js ``` Ela deve usar: ```text BaseStageScene ``` Assim como as fases anteriores. Idealmente deve ser pequena, contendo basicamente: ```text configuração interações posições objetivo ``` Se `Stage3Scene` terminar com centenas de linhas copiadas, a arquitetura regrediu. --- # 17. Configuração da Fase 3 Adicionar ao `stagesData.js`. Algo conceitualmente semelhante a: ```javascript { numero: 3, nome: "Máquina e responsabilidade", objetivo: "Use os equipamentos com responsabilidade", sceneKey: "Stage3Scene" } ``` --- # 18. Cenário simples Ainda podem usar placeholders. Exemplo: ```text ┌──────────────────────────────┐ │ SALA / LABORATÓRIO DE MÁQUINAS │ │ │ PROFESSOR │ │ 👤 │ │ │ │ ⚙ MÁQUINA │ │ │ 👤 JOGADOR │ │ │ 👤 COLEGA │ └──────────────────────────────┘ ``` --- # 19. Interação com a máquina Quando próximo: ```text [E] Interagir ``` Diálogo: ```text A máquina está pronta para a atividade. O que deseja fazer? [Seguir o procedimento correto] [Usar de forma inadequada] ``` --- # 20. Interação com o colega Texto: ```text Um colega propõe usar o equipamento de uma maneira arriscada. O que deseja fazer? [Recusar e alertar sobre o risco] [Aceitar a proposta] ``` --- # 21. Interação com professor/laboratorista Texto: ```text O responsável orienta como utilizar o equipamento com segurança. O que deseja fazer? [Respeitar a orientação] [Ignorar/desrespeitar a orientação] ``` --- # 22. HUD da Fase 3 Reutilizar o mesmo HUD. Mostrar: ```text Objetivo: use os equipamentos com responsabilidade Progresso: 0/3 Cuidado: XX Conduta: XX Danos: XX ``` Agora vale a pena exibir `Danos`, já que ele passa a influenciar o final. --- # 23. Progresso Exatamente como nas outras fases: ```text 0/3 1/3 2/3 3/3 ``` A fase não termina antes de `3/3`. --- # 24. Modal de conclusão Ao chegar a `3/3`: ```text FASE CONCLUÍDA Você concluiu a última etapa da campanha. [FINALIZAR CAMPANHA] ``` Ao clicar: ```javascript ProgressionService.concluirFase(perfil, 3) ``` Depois: ```javascript EndingService.calcularDesfecho(perfil) ``` Depois persistir: ```text faseAtual = 4 fasesConcluidas = [1, 2, 3] campanhaConcluida = true desfecho = ... ``` --- # 25. Criar `EndingScene.js` Novo arquivo: ```text src/scenes/EndingScene.js ``` Essa cena mostra o resultado final da campanha. --- # 26. Final Guardião Exemplo visual: ```text GUARDIÃO DA ESCOLA Suas escolhas demonstraram cuidado, cooperação e responsabilidade. Você ajudou a preservar o ambiente e tornou a escola melhor para todos. Cuidado: XX Conduta: XX Danos: XX [VER FASES] [MENU PRINCIPAL] ``` --- # 27. Final Aprendeu e Reparou ```text APRENDEU E REPAROU Você cometeu alguns erros durante o caminho, mas também tomou decisões responsáveis. As escolhas têm consequências, mas mudar de atitude também faz diferença. Cuidado: XX Conduta: XX Danos: XX ``` --- # 28. Final Responsabilização ```text RESPONSABILIZAÇÃO Algumas escolhas causaram danos e consequências para o ambiente escolar. Cuidar dos espaços e equipamentos é responsabilidade de todos. Cuidado: XX Conduta: XX Danos: XX ``` Evitem uma tela humilhante ou excessivamente punitiva. O objetivo continua sendo educativo. --- # 29. `StageResultScene` da Fase 3 Há duas possibilidades. A mais simples: ```text Stage3 → concluir → EndingScene ``` Então não é necessário passar primeiro por `StageResultScene`. Eu recomendo isso. Fases 1 e 2: ```text StageResultScene ``` Fase 3: ```text EndingScene ``` --- # 30. StageSelect depois da campanha Após a campanha terminar: ```text Fase 1 ✓ Fase 2 ✓ Fase 3 ✓ ``` Também mostrar algo como: ```text Campanha concluída ``` Opcionalmente: ```text Final: Guardião da Escola ``` Mas não precisa exibir detalhes excessivos. --- # 31. Continuar depois da campanha Quando o usuário clicar: ```text CONTINUAR ``` com: ```javascript campanhaConcluida === true ``` pode ir para: ```text StageSelectScene ``` E de lá disponibilizar: ```text VER RESULTADO FINAL ``` ou permitir abrir `EndingScene`. --- # 32. Não conceder pontos repetidos Fases concluídas podem continuar sendo visitadas se vocês mantiverem a lógica atual. Mas: ```text interação já resolvida → ✓ RESOLVIDO → nenhum novo ponto ``` Isso precisa valer também para a Fase 3. --- # Testes do Marco 4 Atualmente temos 25 testes. O Marco 4 deve preservar todos e acrescentar novos. ## 33. Teste 26 — acesso à Fase 3 Perfil: ```text fasesConcluidas = [1] ``` Esperado: ```text Fase 3 = bloqueada ``` Perfil: ```text fasesConcluidas = [1, 2] ``` Esperado: ```text Fase 3 = disponível ``` --- ## 34. Teste 27 — impedir conclusão fora de ordem Tentar: ```javascript concluirFase(perfilNovo, 3) ``` Esperado: ```text falha ``` --- ## 35. Teste 28 — máquina positiva ```text fase3_seguir_procedimento ``` Esperado: ```text +10 cuidado +8 conduta 0 danos ``` --- ## 36. Teste 29 — máquina negativa ```text fase3_usar_maquina_incorretamente ``` Esperado: ```text 0 cuidado -15 conduta +1 dano ``` --- ## 37. Teste 30 — risco positivo ```text fase3_recusar_risco ``` Esperado: ```text +8 cuidado +10 conduta ``` --- ## 38. Teste 31 — risco negativo ```text fase3_aceitar_risco ``` Esperado: ```text -15 conduta +1 dano ``` --- ## 39. Teste 32 — professor Testar ambas as alternativas e exclusão mútua. --- ## 40. Teste 33 — progresso da Fase 3 Confirmar: ```text 0/3 1/3 2/3 3/3 ``` --- ## 41. Teste 34 — conclusão da campanha Depois da Fase 3: ```text faseAtual = 4 fasesConcluidas = [1, 2, 3] campanhaConcluida = true ``` --- ## 42. Testes de finais Criar pelo menos três perfis artificiais. ### Teste 35 ```text conduta = 80 danos = 0 ``` Esperado: ```text guardiao ``` ### Teste 36 ```text conduta = 20 danos = 1 ``` Esperado: ```text reparacao ``` ### Teste 37 ```text conduta = -30 ``` Esperado: ```text responsabilizacao ``` ### Teste 38 ```text conduta = 30 danos = 2 ``` Esperado: ```text responsabilizacao ``` --- ## 43. Teste completo positivo Criar perfil: ```text João Marco 4 ``` Fase 1 positiva: ```text Cuidado = 30 Conduta = 24 ``` Fase 2 positiva: ```text Cuidado = 56 Conduta = 52 ``` Fase 3 positiva: ```text +26 cuidado +28 conduta ``` Resultado final esperado: ```text Cuidado = 82 Conduta = 80 Danos = 0 ``` Final esperado: ```text GUARDIÃO DA ESCOLA ``` --- # 44. Teste misto Criar outro perfil. Completar as três fases com escolhas positivas e negativas. Esperado: ```text 3/3 em cada fase campanha concluída ``` O final deverá depender da pontuação resultante. --- # 45. Teste de responsabilização Criar perfil de teste realizando escolhas negativas suficientes para: ```text danos >= 2 ``` Esperado: ```text RESPONSABILIZAÇÃO ``` --- # 46. Teste de reload Depois do final: ```text F5 ``` Depois: ```text CONTINUAR ``` Deve preservar: ```text pontuação danos escolhas fasesConcluidas campanhaConcluida desfecho ``` --- # 47. Teste com perfis diferentes Exemplo: ```text João → Guardião Maria → Reparação Carlos → Responsabilização ``` Cada perfil deve manter seu próprio final. --- # README Atualizar para: ```text Marco 4 — Fase 3 e conclusão da campanha ``` Documentar: ```text 3 fases jogáveis BaseStageScene StageSelectScene Stage3Scene EndingService EndingScene danos 3 desfechos persistência da campanha testes ``` Não declarar arte final ou áudio completo. --- # O que NÃO fazer no Marco 4 Ainda não façam: ```text*

*arte definitiva completa mapas definitivos áudio completo dublagem ranking online servidor SQL login com senha multiplayer conquistas complexas inventário combate cutscenes elaboradas sistema de nuvem ``` Essas tarefas pertencem a uma etapa posterior de polimento. --- # Checklist final do Marco 4 O Marco 4 só estará aprovado quando: - [ ] Todos os 25 testes anteriores continuarem passando. - [ ] `Stage3Scene` existir. - [ ] `Stage3Scene` reutilizar `BaseStageScene`. - [ ] Fase 3 aparecer na seleção. - [ ] Fase 3 ficar bloqueada antes da Fase 2. - [ ] Concluir Fase 2 desbloquear Fase 3. - [ ] Acesso direto indevido à Fase 3 for bloqueado. - [ ] `stagesData.js` incluir as três fases. - [ ] `StageSelectScene` usar os dados centralizados. - [ ] `ProgressionService` validar sequência. - [ ] A máquina possuir duas escolhas. - [ ] A situação de risco possuir duas escolhas. - [ ] A orientação do professor possuir duas escolhas. - [ ] IDs da Fase 3 forem exclusivos. - [ ] `ChoiceService` atualizar danos. - [ ] O HUD mostrar danos. - [ ] O HUD mostrar objetivo dinâmico da Fase 3. - [ ] O progresso funcionar de `0/3` até `3/3`. - [ ] Interações resolvidas não puderem ser repetidas. - [ ] Fase 3 não terminar antes de `3/3`. - [ ] `faseAtual` se tornar `4`. - [ ] `fasesConcluidas` se tornar `[1, 2, 3]`. - [ ] `campanhaConcluida` ficar `true`. - [ ] `EndingService` existir. - [ ] Final Guardião funcionar. - [ ] Final Aprendeu e Reparou funcionar. - [ ] Final Responsabilização funcionar. - [ ] `EndingScene` existir. - [ ] Final permanecer após reload. - [ ] Perfis independentes poderem ter finais diferentes. - [ ] Saves do Marco 3 continuarem funcionando. - [ ] `npm run test:backend` passar. - [ ] `npm run build` passar. - [ ] `npm run dev` funcionar. - [ ] README estiver atualizado. - [ ] Nenhum erro relevante aparecer no console. Ao terminar o Marco 4, vocês terão o **MVP completo da campanha**: ```text Fase 1 — Laboratório ↓ Fase 2 — Corredor ↓ Fase 3 — Máquina ↓ Histórico das escolhas ↓ Guardião da Escola OU Aprendeu e Reparou OU Responsabilização ``` --- # Developer de Back-end / lógica ## 6. Primeiro aprimoramento: fortalecer `ProgressionService` No Marco 3 identificamos uma pequena fragilidade: era possível chamar diretamente: ```javascript ProgressionService.concluirFase(perfil, 2) ``` sem garantir internamente que a fase anterior estava concluída. Agora corrijam isso. O próprio serviço deve validar a sequência. Exemplo conceitual: ```javascript concluirFase(perfil, numeroFase) ``` deve verificar: ```text Fase 1 → permitida Fase 2 → Fase 1 precisa estar concluída Fase 3 → Fases 1 e 2 precisam estar concluídas ``` Assim, a regra fica protegida mesmo fora da interface. --- # 7. Centralizar informações das fases No Marco 3 foi criado: ```text stagesData.js ``` Agora ele deve ser definitivamente a fonte principal das fases. Evitem manter uma segunda lista hard-coded dentro da `StageSelectScene`. `stagesData.js` deve conter algo equivalente a: ```javascript Fase 1 Laboratório Fase 2 Corredor depois do intervalo Fase 3 Máquina e responsabilidade ``` Incluindo: ```text número nome objetivo sceneKey status ``` quando fizer sentido. --- # 8. Liberar a Fase 3 No Marco 3: ```text Fase 3 = em desenvolvimento ``` Agora a regra muda. Perfil novo: ```text Fase 1 → disponível Fase 2 → bloqueada Fase 3 → bloqueada ``` Depois da Fase 1: ```text Fase 1 → concluída Fase 2 → disponível Fase 3 → bloqueada ``` Depois da Fase 2: ```text Fase 1 → concluída Fase 2 → concluída Fase 3 → disponível ``` Depois da Fase 3: ```text Fase 1 → concluída Fase 2 → concluída Fase 3 → concluída Campanha → concluída ``` --- # 9. Escolhas da Fase 3 Adicionar seis novas escolhas em `choicesData.js`. ### Interação 1 — máquina IDs: ```text fase3_seguir_procedimento fase3_usar_maquina_incorretamente ``` Interação: ```text fase3_maquina_01 ``` Sugestão positiva: ```text +10 cuidado +8 conduta 0 danos ``` Negativa: ```text 0 cuidado -15 conduta +1 dano ``` --- # 10. Interação 2 — proposta arriscada IDs: ```text fase3_recusar_risco fase3_aceitar_risco ``` Interação: ```text fase3_risco_01 ``` Positiva: ```text +8 cuidado +10 conduta 0 danos ``` Negativa: ```text 0 cuidado -15 conduta +1 dano ``` --- # 11. Interação 3 — orientação do responsável IDs: ```text fase3_respeitar_orientacao fase3_desrespeitar_orientacao ``` Interação: ```text fase3_professor_01 ``` Positiva: ```text +8 cuidado +10 conduta 0 danos ``` Negativa: ```text 0 cuidado -10 conduta 0 danos ``` Assim, um caminho totalmente positivo na Fase 3 acrescenta: ```text Cuidado: +26 Conduta: +28 ``` --- # 12. Atualizar `ChoiceService` O `ChoiceService` deve continuar sendo único. Se uma escolha possuir: ```javascript danos: 1 ``` ele deve atualizar: ```javascript perfil.pontuacao.danos ``` Exemplo conceitual: ```javascript perfil.pontuacao.danos += escolha.danos ?? 0; ``` Não calculem dano diretamente na `Stage3Scene`. --- # 13. Criar `EndingService` Novo arquivo recomendado: ```text src/services/EndingService.js ``` Sua função será analisar o perfil completo depois da Fase 3. Exemplo de interface: ```javascript EndingService.calcularDesfecho(perfil) ``` Retorno: ```javascript { id: "guardiao", titulo: "Guardião da Escola", mensagem: "..." } ``` --- # 14. Três desfechos mínimos O GDD prevê três finais principais: **Guardião da Escola**, **Aprendeu e Reparou** e **Responsabilização**. Documento_Projeto_Jogo_Educacio... Para o MVP, adotem uma regra objetiva. ### Guardião da Escola Sugestão: ```text conduta >= 60 E danos <= 1 ``` Resultado: ```text GUARDIÃO DA ESCOLA ``` --- ### Responsabilização Sugestão: ```text conduta < -20 OU danos >= 2 ``` Resultado: ```text RESPONSABILIZAÇÃO ``` --- ### Aprendeu e Reparou Todo o restante: ```text APRENDEU E REPAROU ``` Isso cria um final intermediário para trajetórias mistas. --- # 15. Ordem das regras do final Cuidado com a ordem. Por exemplo: ```javascript if (perfil.pontuacao.danos >= 2 || perfil.pontuacao.conduta < -20) { return RESPONSABILIZACAO; } if (perfil.pontuacao.conduta >= 60) { return GUARDIAO; } return REPARACAO; ``` Depois vocês poderão calibrar esses números. Nesse marco, o mais importante é o sistema funcionar. olá gemini eu te enviei agora a quarta e ultima fase do jogo , com base no tutorial que informa a estrutura dessa fase e a parte lógica (back-end) que me cabe me gere o código completo .*

Response: 1. Dados Centralizados das Fases (`src/data/stagesData.js`)

Fonte única de dados para a `StageSelectScene` e os serviços do back-end, sem listas hardcoded.

```
export const STAGES = [
  {
    numero: 1,
    nome: "Laboratório",
    objetivo: "Cuide do laboratório e dos equipamentos",
    sceneKey: "Stage1Scene",
    interacoes: ["computador_01", "lixo_01", "equipamento_01"]
  },
  {
    numero: 2,
    nome: "Corredor depois do intervalo",
    objetivo: "Mantenha o corredor limpo e seguro para todos",
    sceneKey: "Stage2Scene",
    interacoes: ["fase2_lixo_01", "fase2_liquido_01", "fase2_respeito_01"]
  },
  {
    numero: 3,
    nome: "Máquina e responsabilidade",
    objetivo: "Use os equipamentos com responsabilidade",
    sceneKey: "Stage3Scene",
    interacoes: ["fase3_maquina_01", "fase3_risco_01", "fase3_professor_01"]
  }
];

export function obterDadosFase(numeroFase) {
  return STAGES.find((fase) => fase.numero === numeroFase) || null;
}
```

2. Dicionário Completo de Escolhas (`src/data/choicesData.js`)

Inclusão das 6 escolhas da **Fase 3** com suporte ao campo `danos`.

```
export const CHOICES = {
  // ========================================
  // FASE 1 — Laboratório
  // ========================================
  desligar_computador: {
    id: "desligar_computador",
    interacaoId: "computador_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    danos: 0,
    mensagem: "Você desligou corretamente o equipamento."
  },
  ignorar_computador: {
    id: "ignorar_computador",
    interacaoId: "computador_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -5,
    danos: 0,
    mensagem: "O equipamento continuou ligado consumindo energia sem necessidade."
  },
  recolher_lixo: {
    id: "recolher_lixo",
    interacaoId: "lixo_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    danos: 0,
    mensagem: "Você recolheu o lixo do chão."
  },
  deixar_lixo: {
    id: "deixar_lixo",
    interacaoId: "lixo_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -5,
    danos: 0,
    mensagem: "O lixo foi deixado no chão do laboratório."
  },
  guardar_equipamento: {
    id: "guardar_equipamento",
    interacaoId: "equipamento_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    danos: 0,
    mensagem: "Você guardou o equipamento no local adequado."
  },
  usar_equipamento_incorretamente: {
    id: "usar_equipamento_incorretamente",
    interacaoId: "equipamento_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -8,
    danos: 0,
    mensagem: "O equipamento foi utilizado de forma inadequada."
  },
```

```
  // ========================================
  // FASE 2 — Corredor
  // ========================================
  fase2_recolher_lixo: {
    id: "fase2_recolher_lixo",
    interacaoId: "fase2_lixo_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    danos: 0,
    mensagem: "Você recolheu o lixo do corredor e descartou corretamente."
  },
  fase2_ignorar_lixo: {
    id: "fase2_ignorar_lixo",
    interacaoId: "fase2_lixo_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -5,
    danos: 0,
    mensagem: "O lixo permaneceu largado no meio do corredor."
  },
  fase2_sinalizar_liquido: {
    id: "fase2_sinalizar_liquido",
    interacaoId: "fase2_liquido_01",
    tipo: "positiva",
    cuidado: 8,
    conduta: 10,
    danos: 0,
    mensagem: "Você sinalizou o piso escorregadio e pediu ajuda para a limpeza."
  },
  fase2_ignorar_liquido: {
    id: "fase2_ignorar_liquido",
    interacaoId: "fase2_liquido_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -6,
    danos: 0,
    mensagem: "Você ignorou o líquido derramado, mantendo o risco de quedas."
  },
  fase2_ajudar_colega: {
    id: "fase2_ajudar_colega",
    interacaoId: "fase2_respeito_01",
    tipo: "positiva",
    cuidado: 8,
    conduta: 10,
    danos: 0,
    mensagem: "Você interveio de forma responsável e buscou apoio para mediar o conflito."
  },
  fase2_ignorar_desrespeito: {
    id: "fase2_ignorar_desrespeito",
    interacaoId: "fase2_respeito_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -8,
    danos: 0,
    mensagem: "Você ignorou a situação de desrespeito entre os colegas."
  },

  // ========================================
  // FASE 3 — Máquina e responsabilidade
  // ========================================
  fase3_seguir_procedimento: {
    id: "fase3_seguir_procedimento",
    interacaoId: "fase3_maquina_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    danos: 0,
    mensagem: "Você seguiu o procedimento correto de operação da máquina."
  },
  fase3_usar_maquina_incorretamente: {
    id: "fase3_usar_maquina_incorretamente",
    interacaoId: "fase3_maquina_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -15,
    danos: 1,
    mensagem: "O uso incorreto causou um desgaste severo no equipamento."
  },
  fase3_recusar_risco: {
    id: "fase3_recusar_risco",
    interacaoId: "fase3_risco_01",
    tipo: "positiva",
    cuidado: 8,
    conduta: 10,
    danos: 0,
    mensagem: "Você recusou a proposta arriscada e alertou sobre a segurança."
  },
  fase3_aceitar_risco: {
    id: "fase3_aceitar_risco",
    interacaoId: "fase3_risco_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -15,
    danos: 1,
    mensagem: "A ação arriscada provocou danos ao patrimônio da escola."
  },
  fase3_respeitar_orientacao: {
    id: "fase3_respeitar_orientacao",
    interacaoId: "fase3_professor_01",
    tipo: "positiva",
    cuidado: 8,
    conduta: 10,
    danos: 0,
    mensagem: "Você respeitou as orientações do responsável pela atividade."
  },
  fase3_desrespeitar_orientacao: {
    id: "fase3_desrespeitar_orientacao",
    interacaoId: "fase3_professor_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -10,
    danos: 0,
    mensagem: "Você ignorou as instruções de segurança passadas pelo professor."
  }
};
```

3. Serviço de Regras de Acesso e Progressão (`src/services/ProgressionService.js`)

Implementa validação estrita de sequência, normalização automática de saves antigos e conclusão da campanha ao finalizar a Fase 3.

```
import { STAGES, obterDadosFase } from '../data/stagesData.js';

export class ProgressionService {
  /**
   * Normaliza o estado de progresso de perfis antigos para manter retrocompatibilidade.
   * @param {Object} perfil
   */
  static normalizarPerfil(perfil) {
```

```
    if (!perfil || typeof perfil !== "object") return;

    if (!perfil.progresso) {
      perfil.progresso = { faseAtual: 1, fasesConcluidas: [], campanhaConcluida: false };
    }

    if (!Array.isArray(perfil.progresso.fasesConcluidas)) {
      perfil.progresso.fasesConcluidas = [];
    }

    if (typeof perfil.progresso.campanhaConcluida !== "boolean") {
      perfil.progresso.campanhaConcluida = perfil.progresso.fasesConcluidas.includes(3);
    }

    if (!perfil.pontuacao) {
      perfil.pontuacao = { cuidado: 0, conduta: 0, danos: 0 };
    } else if (typeof perfil.pontuacao.danos !== "number") {
      perfil.pontuacao.danos = 0;
    }
  }

  /**
   * Valida permissão de acesso à fase informada.
   * Exige que todas as fases anteriores estejam na lista 'fasesConcluidas'.
   * @param {Object} perfil
   * @param {number} numeroFase
   * @returns {boolean}
   */
  static podeAcessarFase(perfil, numeroFase) {
    if (!perfil || typeof perfil !== "object") return false;
    this.normalizarPerfil(perfil);

    if (numeroFase === 1) return true;

    const fasesConcluidas = perfil.progresso.fasesConcluidas;

    // Garante validação sequencial encadeada
    for (let i = 1; i < numeroFase; i++) {
      if (!fasesConcluidas.includes(i)) {
        return false;
      }
    }

    return numeroFase <= STAGES.length;
  }

  /**
   * Retorna o status consolidado de uma fase para consumo da UI.
   * @param {Object} perfil
   * @param {number} numeroFase
   * @returns {Object}
   */
  static obterStatusFase(perfil, numeroFase) {
    this.normalizarPerfil(perfil);
    const dadosFase = obterDadosFase(numeroFase);

    if (!dadosFase) {
      return {
        numero: numeroFase,
        status: "em_desenvolvimento",
        disponivel: false,
        concluida: false,
        interacoesTotal: 0
      };
    }

    const estaConcluida = perfil.progresso.fasesConcluidas.includes(numeroFase);
    const disponivel = this.podeAcessarFase(perfil, numeroFase);

    return {
      numero: numeroFase,
      nome: dadosFase.nome,
      objetivo: dadosFase.objetivo,
      sceneKey: dadosFase.sceneKey,
      status: estaConcluida ? "concluida" : (disponivel ? "disponivel" : "bloqueada"),
      disponivel,
      concluida: estaConcluida,
      interacoesTotal: dadosFase.interacoes.length
    };
  }

  /**
   * Conclui uma fase garantindo rigorosamente que as anteriores foram concluídas.
   * @param {Object} perfil
   * @param {number} faseConcluida
   * @returns {Object}
   */
  static concluirFase(perfil, faseConcluida) {
    if (!perfil || typeof perfil !== "object") {
      return { sucesso: false, erro: "Perfil inválido." };
    }

    this.normalizarPerfil(perfil);

    // Impedir conclusão se a fase anterior não tiver sido concluída
    if (faseConcluida > 1) {
      const faseAnteriorConcluida = perfil.progresso.fasesConcluidas.includes(faseConcluida - 1);
      if (!faseAnteriorConcluida) {
        return {
          sucesso: false,
          erro: `Não é possível concluir a Fase ${faseConcluida} sem ter concluído a Fase ${faseConcluida - 1}.`
        };
      }
    }

    if (!perfil.progresso.fasesConcluidas.includes(faseConcluida)) {
      perfil.progresso.fasesConcluidas.push(faseConcluida);
    }

    perfil.progresso.faseAtual = Math.max(
      perfil.progresso.faseAtual || 1,
      faseConcluida + 1
    );

    if (faseConcluida === 3) {
      perfil.progresso.campanhaConcluida = true;
    }

    return {
      sucesso: true,
      perfil
    };
  }
}
```

4. Serviço de Escolhas e Métricas (`src/services/ChoiceService.js`)

Atualizado para contabilizar e persistir o acumulador de danos no perfil do jogador.

```
import { CHOICES } from '../data/choicesData.js';
import { ProgressionService } from './ProgressionService.js';

export class ChoiceService {
  /**
   * Verifica se uma interação já foi resolvida no perfil.
   */
  static interacaoConcluida(perfil, interacaoId) {
    if (!perfil || !Array.isArray(perfil.escolhasRealizadas)) {
      return false;
    }

    return perfil.escolhasRealizadas.some((registro) => {
      const choiceId = typeof registro === "string" ? registro : registro?.choiceId;
      return CHOICES[choiceId]?.interacaoId === interacaoId;
    });
  }

  /**
   * Retorna o progresso de interações de qualquer fase.
   */
  static obterProgressoFase(perfil, listaInteracoes = []) {
    if (!perfil || !Array.isArray(listaInteracoes) || listaInteracoes.length === 0) {
      return { resolvidas: 0, total: 0, texto: "0/0", faseConcluida: false };
    }

    const resolvidas = listaInteracoes.filter((interacaoId) =>
      this.interacaoConcluida(perfil, interacaoId)
    ).length;

    const total = listaInteracoes.length;

    return {
      resolvidas,
      total,
      texto: `${resolvidas}/${total}`,
      faseConcluida: resolvidas === total
    };
  }

  /**
   * Registra a escolha realizada e incrementa cuidado, conduta e danos.
   */
  static registrarEscolha(perfil, choiceId) {
    const escolha = CHOICES[choiceId];

    if (!escolha || !perfil || typeof perfil !== "object") {
      return { sucesso: false, erro: "Escolha inválida ou não encontrada." };
    }

    ProgressionService.normalizarPerfil(perfil);

    if (!Array.isArray(perfil.escolhasRealizadas)) {
      perfil.escolhasRealizadas = [];
    }

    const escolhaJaRegistrada = perfil.escolhasRealizadas.some((registro) => {
      const idRegistrado = typeof registro === "string" ? registro : registro?.choiceId;
      return idRegistrado === choiceId;
    });

    if (escolhaJaRegistrada) {
      return {
        sucesso: false,
        erro: "Esta ação já foi realizada anteriormente.",
        perfil
      };
    }

    if (escolha.interacaoId && this.interacaoConcluida(perfil, escolha.interacaoId)) {
      return {
        sucesso: false,
        erro: "Esta interação já foi resolvida anteriormente.",
        interacaoConcluida: true,
        perfil
      };
    }

    // Atualização de métricas incluindo o campo 'danos'
    perfil.pontuacao.cuidado += escolha.cuidado || 0;
    perfil.pontuacao.conduta += escolha.conduta || 0;
    perfil.pontuacao.danos += escolha.danos ?? 0;

    perfil.escolhasRealizadas.push(choiceId);

    return {
      sucesso: true,
      mensagem: escolha.mensagem,
      impacto: {
        cuidado: escolha.cuidado,
        conduta: escolha.conduta,
        danos: escolha.danos ?? 0
      },
      perfil
    };
  }
}
```

5. Serviço de Cálculo de Desfechos (`src/services/EndingService.js`)

Módulo responsável por avaliar os critérios objetivos de final da campanha e gravar a propriedade `desfecho` no perfil para persistência entre sessões.

```
import { ProgressionService } from './ProgressionService.js';

export const DESFECHOS = {
  RESPONSABILIZACAO: {
    id: "responsabilizacao",
    titulo: "Responsabilização",
    mensagem: "Algumas escolhas causaram danos e consequências para o ambiente escolar. Cuidar dos espaços e equipamentos é responsabilidade de todos."
  },
  GUARDIAO: {
    id: "guardiao",
    titulo: "Guardião da Escola",
    mensagem: "Suas escolhas demonstraram cuidado, cooperação e responsabilidade. Você ajudou a preservar o ambiente e tornou a escola melhor para todos."
  },
  REPARACAO: {
    id: "reparacao",
    titulo: "Aprendeu e Reparou",
    mensagem: "Você cometeu alguns erros durante o caminho, mas também tomou decisões responsáveis. As escolhas têm consequências, mas mudar de atitude também faz diferença."
  }
};

export class EndingService {
  /**
   * Avalia a pontuação acumulada e retorna a estrutura do desfecho final.
   * Regras:
   * 1. Responsabilização: conduta < -20 OU danos >= 2
   * 2. Guardião da Escola: conduta >= 60 E danos <= 1
   * 3. Aprendeu e Reparou: demasia das trajetórias
```

```
   * @param {Object} perfil
   * @returns {Object} Dados do desfecho
   */
  static calcularDesfecho(perfil) {
    if (!perfil) return DESFECHOS.REPARACAO;

    ProgressionService.normalizarPerfil(perfil);

    const conduta = perfil.pontuacao?.conduta ?? 0;
    const danos = perfil.pontuacao?.danos ?? 0;

    // Regra 1: Responsabilização tem prioridade
    if (danos >= 2 || conduta < -20) {
      return DESFECHOS.RESPONSABILIZACAO;
    }

    // Regra 2: Guardião da Escola
    if (conduta >= 60 && danos <= 1) {
      return DESFECHOS.GUARDIAO;
    }

    // Regra 3: Aprendeu e Reparou (trajetórias mistas)
    return DESFECHOS.REPARACAO;
  }

  /**
   * Grava o ID do desfecho diretamente na propriedade do perfil para persistência.
   * @param {Object} perfil
   * @returns {Object} Perfil atualizado com o desfecho fixado
   */
  static aplicarDesfecho(perfil) {
    if (!perfil || typeof perfil !== "object") return null;

    const resultado = this.calcularDesfecho(perfil);
    perfil.desfecho = resultado.id;

    return {
      desfecho: resultado,
      perfil
    };
  }
}
```

6. Testes Automatizados da Lógica (`tests/marco4Backend.test.js`)

Bateria de testes cobrindo os cenários 33 a 47 solicitados na especificação.

```
import { ProgressionService } from '../src/services/ProgressionService.js';
import { ChoiceService } from '../src/services/ChoiceService.js';
import { EndingService, DESFECHOS } from '../src/services/EndingService.js';
import { STAGES } from '../src/data/stagesData.js';

describe('Marco 4 — Testes de Back-end, Regras de Progressão e Desfechos', () => {

  test('Teste 26: Permissão de acesso sequencial à Fase 3', () => {
    const perfilFase1 = { progresso: { faseAtual: 2, fasesConcluidas: [1] } };
    expect(ProgressionService.podeAcessarFase(perfilFase1, 3)).toBe(false);

    const perfilFase2 = { progresso: { faseAtual: 3, fasesConcluidas: [1, 2] } };
    expect(ProgressionService.podeAcessarFase(perfilFase2, 3)).toBe(true);
  });

  test('Teste 27: Impedir conclusão da Fase 3 fora de ordem', () => {
    const perfilNovo = { nome: 'Invalido', progresso: { faseAtual: 1, fasesConcluidas: [] } };
    const res = ProgressionService.concluirFase(perfilNovo, 3);

    expect(res.sucesso).toBe(false);
    expect(perfilNovo.progresso.fasesConcluidas).not.toContain(3);
  });

  test('Teste 28 e 29: Escolhas da Máquina e contabilização de Danos', () => {
    const perfilPos = { pontuacao: { cuidado: 0, conduta: 0, danos: 0 }, escolhasRealizadas: [] };
    ChoiceService.registrarEscolha(perfilPos, 'fase3_seguir_procedimento');
    expect(perfilPos.pontuacao.cuidado).toBe(10);
    expect(perfilPos.pontuacao.conduta).toBe(8);
    expect(perfilPos.pontuacao.danos).toBe(0);

    const perfilNeg = { pontuacao: { cuidado: 0, conduta: 0, danos: 0 }, escolhasRealizadas: [] };
    ChoiceService.registrarEscolha(perfilNeg, 'fase3_usar_maquina_incorretamente');
    expect(perfilNeg.pontuacao.cuidado).toBe(0);
    expect(perfilNeg.pontuacao.conduta).toBe(-15);
    expect(perfilNeg.pontuacao.danos).toBe(1);
  });

  test('Teste 30 e 31: Situação de risco e atualização de danos', () => {
    const perfilPos = { pontuacao: { cuidado: 0, conduta: 0, danos: 0 }, escolhasRealizadas: [] };
    ChoiceService.registrarEscolha(perfilPos, 'fase3_recusar_risco');
    expect(perfilPos.pontuacao.cuidado).toBe(8);
    expect(perfilPos.pontuacao.conduta).toBe(10);

    const perfilNeg = { pontuacao: { cuidado: 0, conduta: 0, danos: 0 }, escolhasRealizadas: [] };
    ChoiceService.registrarEscolha(perfilNeg, 'fase3_aceitar_risco');
    expect(perfilNeg.pontuacao.conduta).toBe(-15);
    expect(perfilNeg.pontuacao.danos).toBe(1);
  });

  test('Teste 33 e 34: Progresso e conclusão total da campanha', () => {
    const perfil = {
      progresso: { faseAtual: 3, fasesConcluidas: [1, 2], campanhaConcluida: false },
      pontuacao: { cuidado: 0, conduta: 0, danos: 0 },
      escolhasRealizadas: []
    };

    const interacoesFase3 = STAGES.find(s => s.numero === 3).interacoes;

    ChoiceService.registrarEscolha(perfil, 'fase3_seguir_procedimento');
    ChoiceService.registrarEscolha(perfil, 'fase3_recusar_risco');
    ChoiceService.registrarEscolha(perfil, 'fase3_respeitar_orientacao');

    const statusProgresso = ChoiceService.obterProgressoFase(perfil, interacoesFase3);
    expect(statusProgresso.texto).toBe('3/3');
    expect(statusProgresso.faseConcluida).toBe(true);

    ProgressionService.concluirFase(perfil, 3);
    expect(perfil.progresso.faseAtual).toBe(4);
    expect(perfil.progresso.fasesConcluidas).toEqual([1, 2, 3]);
    expect(perfil.progresso.campanhaConcluida).toBe(true);
  });

  test('Testes 35, 36, 37 e 38: Regras do EndingService', () => {
    // Guardião
    const p1 = { pontuacao: { conduta: 80, danos: 0, cuidado: 82 } };
    expect(EndingService.calcularDesfecho(p1).id).toBe(DESFECHOS.GUARDIAO.id);

    // Aprendeu e Reparou
    const p2 = { pontuacao: { conduta: 20, danos: 1, cuidado: 30 } };
    expect(EndingService.calcularDesfecho(p2).id).toBe(DESFECHOS.REPARACAO.id);

    // Responsabilização por conduta negativa
```

```
    const p3 = { pontuacao: { conduta: -30, danos: 0, cuidado: 10 } };
    expect(EndingService.calcularDesfecho(p3).id).toBe(DESFECHOS.RESPONSABILIZACAO.id);

    // Responsabilização por acúmulo de danos
    const p4 = { pontuacao: { conduta: 30, danos: 2, cuidado: 20 } };
    expect(EndingService.calcularDesfecho(p4).id).toBe(DESFECHOS.RESPONSABILIZACAO.id);
  });

  test('Teste 43: Trajetória 100% positiva resulta em Guardião da Escola', () => {
    const perfilCompleto = {
      nome: 'João Marco 4',
      pontuacao: { cuidado: 0, conduta: 0, danos: 0 },
      escolhasRealizadas: [],
      progresso: { faseAtual: 1, fasesConcluidas: [], campanhaConcluida: false }
    };

    // Fase 1 Positiva
    ChoiceService.registrarEscolha(perfilCompleto, 'desligar_computador');
    ChoiceService.registrarEscolha(perfilCompleto, 'recolher_lixo');
    ChoiceService.registrarEscolha(perfilCompleto, 'guardar_equipamento');
    ProgressionService.concluirFase(perfilCompleto, 1);

    // Fase 2 Positiva
    ChoiceService.registrarEscolha(perfilCompleto, 'fase2_recolher_lixo');
    ChoiceService.registrarEscolha(perfilCompleto, 'fase2_sinalizar_liquido');
    ChoiceService.registrarEscolha(perfilCompleto, 'fase2_ajudar_colega');
    ProgressionService.concluirFase(perfilCompleto, 2);

    // Fase 3 Positiva
    ChoiceService.registrarEscolha(perfilCompleto, 'fase3_seguir_procedimento');
    ChoiceService.registrarEscolha(perfilCompleto, 'fase3_recusar_risco');
    ChoiceService.registrarEscolha(perfilCompleto, 'fase3_respeitar_orientacao');
    ProgressionService.concluirFase(perfilCompleto, 3);

    EndingService.aplicarDesfecho(perfilCompleto);

    expect(perfilCompleto.pontuacao.cuidado).toBe(82);
    expect(perfilCompleto.pontuacao.conduta).toBe(80);
    expect(perfilCompleto.pontuacao.danos).toBe(0);
    expect(perfilCompleto.desfecho).toBe('guardiao');
    expect(perfilCompleto.progresso.campanhaConcluida).toBe(true);
  });

  test('Teste 46: Persistência do desfecho em reload fictício', () => {
    const perfilOrigem = {
      nome: 'Carlos',
      pontuacao: { cuidado: 40, conduta: -25, danos: 2 },
      escolhasRealizadas: ['fase3_usar_maquina_incorretamente', 'fase3_aceitar_risco'],
      progresso: { faseAtual: 4, fasesConcluidas: [1, 2, 3], campanhaConcluida: true }
    };

    EndingService.aplicarDesfecho(perfilOrigem);

    // Simulação do reload convertendo para JSON e restaurando
    const jsonSaved = JSON.stringify(perfilOrigem);
    const perfilRestaurado = JSON.parse(jsonSaved);

    ProgressionService.normalizarPerfil(perfilRestaurado);

    expect(perfilRestaurado.desfecho).toBe('responsabilizacao');
    expect(perfilRestaurado.progresso.campanhaConcluida).toBe(true);
    expect(perfilRestaurado.pontuacao.danos).toBe(2);
  });
});
```

---

*User prompt: const FASE1_INTERACOES = [{ id: "computador_01", visual: "computador", label: "COMPUTADOR", x: 950, y: 380, raioInteracao: 135, titulo: "Computador", mensagem: "O computador está ligado sem necessidade.\nO que deseja fazer?", opcoes: [{ texto: "Desligar corretamente", choiceId: "desligar_computador" }, { texto: "Ignorar", choiceId: "ignorar_computador" }] }, { id: "lixo_01", visual: "lixo", label: "LIXO", x: 430, y: 530, raioInteracao: 120, titulo: "Lixo no laboratório", mensagem: "Há lixo no chão do laboratório.\nO que deseja fazer?", opcoes: [{ texto: "Colocar na lixeira", choiceId: "recolher_lixo" }, { texto: "Deixar onde está", choiceId: "deixar_lixo" }] }, { id: "equipamento_01", visual: "equipamento", label: "EQUIPAMENTO", x: 760, y: 265, raioInteracao: 120, titulo: "Equipamento fora do lugar", mensagem: "Um equipamento foi deixado fora do lugar.\nO que deseja fazer?", opcoes: [{ texto: "Guardar corretamente", choiceId: "guardar_equipamento" }, { texto: "Usar/deixar de forma inadequada", choiceId: "usar_equipamento_incorretamente" }] }]; const FASE2_INTERACOES = [{ id: "fase2_lixo_01", visual: "lixo", label: "LIXO", x: 330, y: 285, raioInteracao: 120, titulo: "Lixo no corredor", mensagem: "Há lixo no corredor.\nO que deseja fazer?", opcoes: [{ texto: "Recolher e descartar corretamente", choiceId: "fase2_recolher_lixo" }, { texto: "Ignorar", choiceId: "fase2_ignorar_lixo" }] }, { id: "fase2_liquido_01", visual: "liquido", label: "LÍQUIDO", x: 660, y: 520, raioInteracao: 125, titulo: "Líquido derramado", mensagem: "Há líquido derramado no chão.\nIsso pode causar um acidente.", opcoes: [{ texto: "Sinalizar e pedir ajuda", choiceId: "fase2_sinalizar_liquido" }, { texto: "Ignorar", choiceId: "fase2_ignorar_liquido" }] }, { id: "fase2_respeito_01", visual: "respeito", label: "COLEGAS", x: 1000, y: 335, raioInteracao: 135, titulo: "Respeito entre colegas", mensagem: "Você percebe uma situação de desrespeito entre colegas.", opcoes: [{ texto: "Buscar ajuda / agir de forma responsável", choiceId: "fase2_ajudar_colega" }, { texto: "Ignorar", choiceId: "fase2_ignorar_desrespeito" }] }]; export const FASE1_CONFIG = { numeroFase: 1, nome: "Laboratório", cabecalho: "FASE 1 • LABORATÓRIO", objetivo: "cuide do laboratório", mensagemConclusao: "Você analisou todas as situações do laboratório.", jogadorInicial: { x: 230, y: 380 }, sala: { backgroundColor: 0x163b4d, gridColor: 0x2b5f70, borderColor: 0x7ed6df, destaque: "ÁREA DE\nTRABALHO", destaqueX: 180, destaqueY: 255 }, interacoesObrigatorias: ["computador_01", "lixo_01", "equipamento_01"], interacoes: FASE1_INTERACOES }; export const FASE2_CONFIG = { numeroFase: 2, nome: "Corredor depois do intervalo", cabecalho: "FASE 2 • CORREDOR DEPOIS DO INTERVALO", objetivo: "ajude a tornar o corredor seguro", mensagemConclusao: "Você analisou todas as situações do corredor.", jogadorInicial: { x: 230, y: 430 }, sala: { backgroundColor: 0x254052, gridColor: 0x3a6371, borderColor: 0x8ad9cf, destaque: "CORREDOR\nSEGURO", destaqueX: 180, destaqueY: 255, linhasHorizontais: true }, interacoesObrigatorias: ["fase2_lixo_01", "fase2_liquido_01", "fase2_respeito_01"], interacoes: FASE2_INTERACOES }; export const STAGE_CONFIGS = { 1: FASE1_CONFIG, 2: FASE2_CONFIG }; export function obterConfiguracaoFase(numeroFase) { return STAGE_CONFIGS[numeroFase] || null; } export const CHOICES = { // Interação 1: Computador desligar_computador: { id: "desligar_computador", interacaoId: "computador_01", tipo: "positiva", cuidado: 10, conduta: 8, mensagem: "Você desligou corretamente o equipamento." }, ignorar_computador: { id: "ignorar_computador", interacaoId: "computador_01", tipo: "negativa", cuidado: 0, conduta: -5, mensagem: "O equipamento continuou ligado consumindo energia sem necessidade." }, // Interação 2: Lixo recolher_lixo: { id: "recolher_lixo", interacaoId: "lixo_01", tipo: "positiva", cuidado: 10, conduta: 8, mensagem: "Você recolheu o lixo e manteve o ambiente limpo." }, deixar_lixo: { id: "deixar_lixo", interacaoId: "lixo_01", tipo: "negativa", cuidado: 0, conduta: -5, mensagem: "O lixo foi deixado no chão do laboratório." }, // Interação 3: Equipamento guardar_equipamento: { id: "guardar_equipamento", interacaoId: "equipamento_01", tipo: "positiva", cuidado: 10, conduta: 8, mensagem: "Você guardou o equipamento no local adequado." }, usar_equipamento_incorretamente: { id: "usar_equipamento_incorretamente", interacaoId: "equipamento_01", tipo: "negativa", cuidado: 0, conduta: -8, mensagem: "O equipamento foi utilizado de forma inadequada." }, // Interação 4: Lixo no corredor da Fase 2 fase2_recolher_lixo: { id: "fase2_recolher_lixo", interacaoId: "fase2_lixo_01", tipo: "positiva", cuidado: 10, conduta: 8, mensagem: "Você recolheu o lixo e ajudou a manter o corredor seguro." }, fase2_ignorar_lixo: { id: "fase2_ignorar_lixo", interacaoId: "fase2_lixo_01", tipo: "negativa", cuidado: 0, conduta: -5, mensagem: "O lixo continuou no corredor, aumentando o risco de acidentes." }, // Interação 5: Líquido derramado na Fase 2 fase2_sinalizar_liquido: { id: "fase2_sinalizar_liquido", interacaoId: "fase2_liquido_01", tipo: "positiva", cuidado: 8, conduta: 10, mensagem: "Você sinalizou o risco e buscou ajuda para evitar um acidente." }, fase2_ignorar_liquido: { id: "fase2_ignorar_liquido", interacaoId: "fase2_liquido_01", tipo: "negativa", cuidado: 0, conduta: -6, mensagem: "O líquido permaneceu no chão sem qualquer sinalização." }, // Interação 6: Respeito entre colegas na Fase 2 fase2_ajudar_colega: { id: "fase2_ajudar_colega", interacaoId: "fase2_respeito_01", tipo: "positiva", cuidado: 8, conduta: 10, mensagem: "Você buscou ajuda e agiu de forma responsável com os colegas." }, fase2_ignorar_desrespeito: { id: "fase2_ignorar_desrespeito", interacaoId: "fase2_respeito_01", tipo: "negativa", cuidado: 0, conduta: -8, mensagem: "A situação de desrespeito foi ignorada e continuou sem apoio." }, }; export class ProgressionService { /** * Verifica se o perfil pode entrar em uma fase jogável neste marco. * A regra fica no serviço para que a interface não seja a única barreira. */ static podeAcessarFase(perfil, numeroFase) { if (numeroFase === 1) { return true; } if (numeroFase === 2) { return Boolean(perfil?.progresso?.fasesConcluidas?.includes(1)); } // A Fase 3 aparece na seleção, mas ainda não possui gameplay. return false; } /** * Retorna o estado que a tela de seleção deve apresentar para a fase. */ static obterStatusFase(perfil, numeroFase) { if (numeroFase === 3) { return "em_desenvolvimento"; } if (perfil?.progresso?.fasesConcluidas?.includes(numeroFase)) { return "concluida"; } return this.podeAcessarFase(perfil, numeroFase) ? "disponivel" : "bloqueada"; } /** * Conclui a fase atual e atualiza a fase ativa no perfil. * @param {Object} perfil Objeto do perfil atual * @param {number} faseConcluida Número da fase * @returns {Object} Perfil atualizado */ static concluirFase(perfil, faseConcluida) { if (!perfil || typeof perfil !== "object") { return { sucesso: false, erro: "Perfil inválido." }; } if (!perfil.progresso) { perfil.progresso = { faseAtual: 1, fasesConcluidas: [] }; } if (!Array.isArray(perfil.progresso.fasesConcluidas)) { perfil.progresso.fasesConcluidas = []; } if (!perfil.progresso.fasesConcluidas.includes(faseConcluida)) { perfil.progresso.fasesConcluidas.push(faseConcluida); } // Atualiza para a próxima fase mantendo a maior fase alcançada perfil.progresso.faseAtual = Math.max(perfil.progresso.faseAtual || 1, faseConcluida + 1); return perfil; } } import { CHOICES } from '../data/choicesData.js'; export class ChoiceService { /** * Verifica se uma interação específica já foi concluída no perfil. */ static interacaoConcluida(perfil, interacaoId) { if (!perfil || !Array.isArray(perfil.escolhasRealizadas)) { return false; } return perfil.escolhasRealizadas.some((registro) => { const choiceId = typeof registro === "string" ? registro : registro?.choiceId; return CHOICES[choiceId]?.interacaoId === interacaoId; }); } /** * Retorna o progresso atual das interações obrigatorias da fase (ex: "1/3", "3/3") * e indica se todas foram resolvidas. */ static obterProgressoFase(perfil, listaInteracoesObrigatorias = []) { if (!perfil || !Array.isArray(listaInteracoesObrigatorias) || listaInteracoesObrigatorias.length === 0) { return { resolvidas: 0, total: 0, texto: "0/0", faseConcluida: false }; } const resolvidas = listaInteracoesObrigatorias.filter((interacaoId) => this.interacaoConcluida(perfil, interacaoId)).length; const total = listaInteracoesObrigatorias.length; return { resolvidas, total, texto: `${resolvidas}/${total}`, faseConcluida: resolvidas === total }; } /** * Processa a escolha feita pelo jogador e atualiza os dados do perfil. */ static registrarEscolha(perfil, choiceId) { const escolha = CHOICES[choiceId]; if (!escolha || !perfil || typeof perfil !== "object") { return { sucesso: false, erro: "Escolha inválida ou não encontrada." }; } if (!Array.isArray(perfil.escolhasRealizadas)) { perfil.escolhasRealizadas = []; } if (!perfil.pontuacao) { perfil.pontuacao = { cuidado: 0, conduta: 0 }; } // Impede a mesma escolha exata duas vezes const escolhaJaRegistrada = perfil.escolhasRealizadas.some((registro) => { const idRegistrado = typeof registro === "string" ? registro : registro?.choiceId; return idRegistrado === choiceId; }); if (escolhaJaRegistrada) { return { sucesso: false, erro: "Esta ação já foi realizada anteriormente.", perfil }; } // Impede responder a mesma interação mais de uma vez if (escolha.interacaoId && this.interacaoConcluida(perfil, escolha.interacaoId)) { return { sucesso: false, erro: "Esta interação já foi resolvida anteriormente.", interacaoConcluida: true, perfil }; } // Aplicação das consequências na pontuação perfil.pontuacao.cuidado = (perfil.pontuacao.cuidado || 0) + escolha.cuidado; perfil.pontuacao.conduta = (perfil.pontuacao.conduta || 0) + escolha.conduta; // Registrar ação concluída perfil.escolhasRealizadas.push(choiceId); return { sucesso: true, mensagem: escolha.mensagem, impacto: { cuidado: escolha.cuidado, conduta: escolha.conduta }, perfil }; } } eu preciso que voce gere o codigo completo somente desses 4 primeiros*

Response: 1. `src/data/stagesData.js`

```
const FASE1_INTERACOES = [
  {
    id: "computador_01",
    visual: "computador",
    label: "COMPUTADOR",
    x: 950,
    y: 380,
    raioInteracao: 135,
```

```
    titulo: "Computador",
    mensagem: "O computador está ligado sem necessidade.\nO que deseja fazer?",
    opcoes: [
      { texto: "Desligar corretamente", choiceId: "desligar_computador" },
      { texto: "Ignorar", choiceId: "ignorar_computador" }
    ]
  },
  {
    id: "lixo_01",
    visual: "lixo",
    label: "LIXO",
    x: 430,
    y: 530,
    raioInteracao: 120,
    titulo: "Lixo no laboratório",
    mensagem: "Há lixo no chão do laboratório.\nO que deseja fazer?",
    opcoes: [
      { texto: "Colocar na lixeira", choiceId: "recolher_lixo" },
      { texto: "Deixar onde está", choiceId: "deixar_lixo" }
    ]
  },
  {
    id: "equipamento_01",
    visual: "equipamento",
    label: "EQUIPAMENTO",
    x: 760,
    y: 265,
    raioInteracao: 120,
    titulo: "Equipamento fora do lugar",
    mensagem: "Um equipamento foi deixado fora do lugar.\nO que deseja fazer?",
    opcoes: [
      { texto: "Guardar corretamente", choiceId: "guardar_equipamento" },
      { texto: "Usar/deixar de forma inadequada", choiceId: "usar_equipamento_incorretamente" }
    ]
  }
];

const FASE2_INTERACOES = [
  {
    id: "fase2_lixo_01",
    visual: "lixo",
    label: "LIXO",
    x: 330,
    y: 285,
    raioInteracao: 120,
    titulo: "Lixo no corredor",
    mensagem: "Há lixo no corredor.\nO que deseja fazer?",
    opcoes: [
      { texto: "Recolher e descartar corretamente", choiceId: "fase2_recolher_lixo" },
      { texto: "Ignorar", choiceId: "fase2_ignorar_lixo" }
    ]
  },
  {
    id: "fase2_liquido_01",
    visual: "liquido",
    label: "LÍQUIDO",
    x: 660,
    y: 520,
    raioInteracao: 125,
    titulo: "Líquido derramado",
    mensagem: "Há líquido derramado no chão.\nIsso pode causar um acidente.",
    opcoes: [
      { texto: "Sinalizar e pedir ajuda", choiceId: "fase2_sinalizar_liquido" },
      { texto: "Ignorar", choiceId: "fase2_ignorar_liquido" }
    ]
  },
  {
    id: "fase2_respeito_01",
    visual: "respeito",
    label: "COLEGAS",
    x: 1000,
    y: 335,
    raioInteracao: 135,
    titulo: "Respeito entre colegas",
    mensagem: "Você percebe uma situação de desrespeito entre colegas.",
    opcoes: [
      { texto: "Buscar ajuda / agir de forma responsável", choiceId: "fase2_ajudar_colega" },
      { texto: "Ignorar", choiceId: "fase2_ignorar_desrespeito" }
    ]
  }
];

const FASE3_INTERACOES = [
  {
    id: "fase3_maquina_01",
    visual: "maquina",
    label: "MÁQUINA",
    x: 850,
    y: 380,
    raioInteracao: 135,
    titulo: "Uso de Equipamento / Máquina",
    mensagem: "A máquina está pronta para a atividade.\nO que deseja fazer?",
    opcoes: [
      { texto: "Seguir o procedimento correto", choiceId: "fase3_seguir_procedimento" },
      { texto: "Usar de forma inadequada", choiceId: "fase3_usar_maquina_incorretamente" }
    ]
  },
  {
    id: "fase3_risco_01",
    visual: "colega",
    label: "COLEGA",
    x: 450,
    y: 520,
    raioInteracao: 125,
    titulo: "Proposta Arriscada",
    mensagem: "Um colega propõe usar o equipamento de uma maneira arriscada.\nO que deseja fazer?",
    opcoes: [
      { texto: "Recusar e alertar sobre o risco", choiceId: "fase3_recusar_risco" },
      { texto: "Aceitar a proposta", choiceId: "fase3_aceitar_risco" }
    ]
  },
  {
    id: "fase3_professor_01",
    visual: "professor",
    label: "PROFESSOR",
    x: 300,
    y: 280,
    raioInteracao: 130,
    titulo: "Orientação do Responsável",
    mensagem: "O responsável orienta como utilizar o equipamento com segurança.\nO que deseja fazer?",
    opcoes: [
      { texto: "Respeitar a orientação", choiceId: "fase3_respeitar_orientacao" },
      { texto: "Ignorar/desrespeitar a orientação", choiceId: "fase3_desrespeitar_orientacao" }
    ]
  }
];

export const FASE1_CONFIG = {
  numeroFase: 1,
  nome: "Laboratório",
```

```
  cabecalho: "FASE 1  •  LABORATÓRIO",
  objetivo: "cuide do laboratório",
  mensagemConclusao: "Você analisou todas as situações do laboratório.",
  jogadorInicial: { x: 230, y: 380 },
  sala: {
    backgroundColor: 0x163b4d,
    gridColor: 0x2b5f70,
    borderColor: 0x7ed6df,
    destaque: "ÁREA DE\nTRABALHO",
    destaqueX: 180,
    destaqueY: 255
  },
  interacoesObrigatorias: ["computador_01", "lixo_01", "equipamento_01"],
  interacoes: FASE1_INTERACOES
};

export const FASE2_CONFIG = {
  numeroFase: 2,
  nome: "Corredor depois do intervalo",
  cabecalho: "FASE 2  •  CORREDOR DEPOIS DO INTERVALO",
  objetivo: "ajude a tornar o corredor seguro",
  mensagemConclusao: "Você analisou todas as situações do corredor.",
  jogadorInicial: { x: 230, y: 430 },
  sala: {
    backgroundColor: 0x254052,
    gridColor: 0x3a6371,
    borderColor: 0x8ad9cf,
    destaque: "CORREDOR\nSEGURO",
    destaqueX: 180,
    destaqueY: 255,
    linhasHorizontais: true
  },
  interacoesObrigatorias: [
    "fase2_lixo_01",
    "fase2_liquido_01",
    "fase2_respeito_01"
  ],
  interacoes: FASE2_INTERACOES
};

export const FASE3_CONFIG = {
  numeroFase: 3,
  nome: "Máquina e responsabilidade",
  cabecalho: "FASE 3  •  MÁQUINA E RESPONSABILIDADE",
  objetivo: "use os equipamentos com responsabilidade",
  mensagemConclusao: "Você analisou todas as situações de responsabilidade com os equipamentos.",
  jogadorInicial: { x: 230, y: 380 },
  sala: {
    backgroundColor: 0x1d2d3a,
    gridColor: 0x30485c,
    borderColor: 0x82b1ff,
    destaque: "SALA DE\nMÁQUINAS",
    destaqueX: 180,
    destaqueY: 255
  },
  interacoesObrigatorias: [
    "fase3_maquina_01",
    "fase3_risco_01",
    "fase3_professor_01"
  ],
  interacoes: FASE3_INTERACOES
};

export const STAGE_CONFIGS = {
  1: FASE1_CONFIG,
  2: FASE2_CONFIG,
  3: FASE3_CONFIG
};

export function obterConfiguracaoFase(numeroFase) {
  return STAGE_CONFIGS[numeroFase] || null;
}
```

2. src/data/choicesData.js

```
export const CHOICES = {
  // ==========================================
  // FASE 1 — Laboratório
  // ==========================================
  desligar_computador: {
    id: "desligar_computador",
    interacaoId: "computador_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    danos: 0,
    mensagem: "Você desligou corretamente o equipamento."
  },
  ignorar_computador: {
    id: "ignorar_computador",
    interacaoId: "computador_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -5,
    danos: 0,
    mensagem: "O equipamento continuou ligado consumindo energia sem necessidade."
  },
  recolher_lixo: {
    id: "recolher_lixo",
    interacaoId: "lixo_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    danos: 0,
    mensagem: "Você recolheu o lixo e manteve o ambiente limpo."
  },
  deixar_lixo: {
    id: "deixar_lixo",
    interacaoId: "lixo_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -5,
    danos: 0,
    mensagem: "O lixo foi deixado no chão do laboratório."
  },
  guardar_equipamento: {
    id: "guardar_equipamento",
    interacaoId: "equipamento_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    danos: 0,
    mensagem: "Você guardou o equipamento no local adequado."
  },
  usar_equipamento_incorretamente: {
    id: "usar_equipamento_incorretamente",
    interacaoId: "equipamento_01",
    tipo: "negativa",
    cuidado: 0,
```

```
    conduta: -8,
    danos: 0,
    mensagem: "O equipamento foi utilizado de forma inadequada."
  },

  // ==========================================
  // FASE 2 — Corredor
  // ==========================================
  fase2_recolher_lixo: {
    id: "fase2_recolher_lixo",
    interacaoId: "fase2_lixo_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    danos: 0,
    mensagem: "Você recolheu o lixo e ajudou a manter o corredor seguro."
  },
  fase2_ignorar_lixo: {
    id: "fase2_ignorar_lixo",
    interacaoId: "fase2_lixo_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -5,
    danos: 0,
    mensagem: "O lixo continuou no corredor, aumentando o risco de acidentes."
  },
  fase2_sinalizar_liquido: {
    id: "fase2_sinalizar_liquido",
    interacaoId: "fase2_liquido_01",
    tipo: "positiva",
    cuidado: 8,
    conduta: 10,
    danos: 0,
    mensagem: "Você sinalizou o risco e buscou ajuda para evitar um acidente."
  },
  fase2_ignorar_liquido: {
    id: "fase2_ignorar_liquido",
    interacaoId: "fase2_liquido_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -6,
    danos: 0,
    mensagem: "O líquido permaneceu no chão sem qualquer sinalização."
  },
  fase2_ajudar_colega: {
    id: "fase2_ajudar_colega",
    interacaoId: "fase2_respeito_01",
    tipo: "positiva",
    cuidado: 8,
    conduta: 10,
    danos: 0,
    mensagem: "Você buscou ajuda e agiu de forma responsável com os colegas."
  },
  fase2_ignorar_desrespeito: {
    id: "fase2_ignorar_desrespeito",
    interacaoId: "fase2_respeito_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -8,
    danos: 0,
    mensagem: "A situação de desrespeito foi ignorada e continuou sem apoio."
  },

  // ==========================================
  // FASE 3 — Máquina e responsabilidade
  // ==========================================
  fase3_seguir_procedimento: {
    id: "fase3_seguir_procedimento",
    interacaoId: "fase3_maquina_01",
    tipo: "positiva",
    cuidado: 10,
    conduta: 8,
    danos: 0,
    mensagem: "Você seguiu o procedimento correto de operação da máquina."
  },
  fase3_usar_maquina_incorretamente: {
    id: "fase3_usar_maquina_incorretamente",
    interacaoId: "fase3_maquina_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -15,
    danos: 1,
    mensagem: "O uso incorreto causou um desgaste severo no equipamento."
  },
  fase3_recusar_risco: {
    id: "fase3_recusar_risco",
    interacaoId: "fase3_risco_01",
    tipo: "positiva",
    cuidado: 8,
    conduta: 10,
    danos: 0,
    mensagem: "Você recusou a proposta arriscada e alertou sobre a segurança."
  },
  fase3_aceitar_risco: {
    id: "fase3_aceitar_risco",
    interacaoId: "fase3_risco_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -15,
    danos: 1,
    mensagem: "A ação arriscada provocou danos ao patrimônio da escola."
  },
  fase3_respeitar_orientacao: {
    id: "fase3_respeitar_orientacao",
    interacaoId: "fase3_professor_01",
    tipo: "positiva",
    cuidado: 8,
    conduta: 10,
    danos: 0,
    mensagem: "Você respeitou as orientações do responsável pela atividade."
  },
  fase3_desrespeitar_orientacao: {
    id: "fase3_desrespeitar_orientacao",
    interacaoId: "fase3_professor_01",
    tipo: "negativa",
    cuidado: 0,
    conduta: -10,
    danos: 0,
    mensagem: "Você ignorou as instruções de segurança passadas pelo professor."
  }
};
```

3. src/services/ProgressionService.js

```
import { STAGE_CONFIGS } from '../data/stagesData.js';

export class ProgressionService {
  /**
   * Normaliza o estado de progresso do perfil para garantir compatibilidade com saves antigos.
```

```
   * @param {Object} perfil
   */
  static normalizarPerfil(perfil) {
    if (!perfil || typeof perfil !== "object") return;

    if (!perfil.progresso) {
      perfil.progresso = { faseAtual: 1, fasesConcluidas: [], campanhaConcluida: false };
    }

    if (!Array.isArray(perfil.progresso.fasesConcluidas)) {
      perfil.progresso.fasesConcluidas = [];
    }

    if (typeof perfil.progresso.campanhaConcluida !== "boolean") {
      perfil.progresso.campanhaConcluida = perfil.progresso.fasesConcluidas.includes(3);
    }

    if (!perfil.pontuacao) {
      perfil.pontuacao = { cuidado: 0, conduta: 0, danos: 0 };
    } else if (typeof perfil.pontuacao.danos !== "number") {
      perfil.pontuacao.danos = 0;
    }
  }

  /**
   * Verifica se o perfil pode acessar uma fase jogável de forma estritamente sequencial.
   * @param {Object} perfil
   * @param {number} numeroFase
   * @returns {boolean}
   */
  static podeAcessarFase(perfil, numeroFase) {
    if (!perfil || typeof perfil !== "object") return false;
    this.normalizarPerfil(perfil);

    if (numeroFase === 1) return true;

    const fasesConcluidas = perfil.progresso.fasesConcluidas;

    for (let i = 1; i < numeroFase; i++) {
      if (!fasesConcluidas.includes(i)) {
        return false;
      }
    }

    return Boolean(STAGE_CONFIGS[numeroFase]);
  }

  /**
   * Retorna o status de apresentação para a StageSelectScene ("concluida" | "disponivel" | "bloqueada" | "em_desenvolvimento").
   * @param {Object} perfil
   * @param {number} numeroFase
   * @returns {string}
   */
  static obterStatusFase(perfil, numeroFase) {
    this.normalizarPerfil(perfil);

    if (!STAGE_CONFIGS[numeroFase]) {
      return "em_desenvolvimento";
    }

    if (perfil.progresso.fasesConcluidas.includes(numeroFase)) {
      return "concluida";
    }

    return this.podeAcessarFase(perfil, numeroFase)
      ? "disponivel"
      : "bloqueada";
  }

  /**
   * Conclui a fase atual garantindo a ordem cronológica, adicionando ao histórico e atualizando a campanha.
   * @param {Object} perfil Objeto do perfil atual
   * @param {number} faseConcluida Número da fase
   * @returns {Object} Perfil atualizado ou objeto de erro
   */
  static concluirFase(perfil, faseConcluida) {
    if (!perfil || typeof perfil !== "object") {
      return { sucesso: false, erro: "Perfil inválido." };
    }

    this.normalizarPerfil(perfil);

    if (faseConcluida > 1) {
      const faseAnteriorConcluida = perfil.progresso.fasesConcluidas.includes(faseConcluida - 1);
      if (!faseAnteriorConcluida) {
        return {
          sucesso: false,
          erro: `Não é possível concluir a Fase ${faseConcluida} sem ter concluído a Fase ${faseConcluida - 1}.`
        };
      }
    }

    if (!perfil.progresso.fasesConcluidas.includes(faseConcluida)) {
      perfil.progresso.fasesConcluidas.push(faseConcluida);
    }

    perfil.progresso.faseAtual = Math.max(
      perfil.progresso.faseAtual || 1,
      faseConcluida + 1
    );

    if (faseConcluida === 3) {
      perfil.progresso.campanhaConcluida = true;
    }

    return perfil;
  }
}
```

4. src/services/ChoiceService.js

```
import { CHOICES } from '../data/choicesData.js';
import { ProgressionService } from './ProgressionService.js';

export class ChoiceService {
  /**
   * Verifica se uma interação específica já foi concluída no perfil.
   */
  static interacaoConcluida(perfil, interacaoId) {
    if (!perfil || !Array.isArray(perfil.escolhasRealizadas)) {
      return false;
    }

    return perfil.escolhasRealizadas.some((registro) => {
      const choiceId = typeof registro === "string" ? registro : registro?.choiceId;
      return CHOICES[choiceId]?.interacaoId === interacaoId;
    });
  }
```

```
  /**
   * Retorna o progresso atual das interações obrigatórias da fase (ex: "1/3", "3/3")
   * e indica se todas foram resolvidas.
   */
  static obterProgressoFase(perfil, listaInteracoesObrigatorias = []) {
    if (!perfil || !Array.isArray(listaInteracoesObrigatorias) || listaInteracoesObrigatorias.length === 0) {
      return { resolvidas: 0, total: 0, texto: "0/0", faseConcluida: false };
    }

    const resolvidas = listaInteracoesObrigatorias.filter((interacaoId) =>
      this.interacaoConcluida(perfil, interacaoId)
    ).length;

    const total = listaInteracoesObrigatorias.length;

    return {
      resolvidas,
      total,
      texto: `${resolvidas}/${total}`,
      faseConcluida: resolvidas === total
    };
  }

  /**
   * Processa a escolha feita pelo jogador e atualiza os dados do perfil (incluindo cuidado, conduta e danos).
   */
  static registrarEscolha(perfil, choiceId) {
    const escolha = CHOICES[choiceId];

    if (!escolha || !perfil || typeof perfil !== "object") {
      return { sucesso: false, erro: "Escolha inválida ou não encontrada." };
    }

    ProgressionService.normalizarPerfil(perfil);

    if (!Array.isArray(perfil.escolhasRealizadas)) {
      perfil.escolhasRealizadas = [];
    }

    const escolhaJaRegistrada = perfil.escolhasRealizadas.some((registro) => {
      const idRegistrado = typeof registro === "string" ? registro : registro?.choiceId;
      return idRegistrado === choiceId;
    });

    if (escolhaJaRegistrada) {
      return {
        sucesso: false,
        erro: "Esta ação já foi realizada anteriormente.",
        perfil
      };
    }

    if (escolha.interacaoId && this.interacaoConcluida(perfil, escolha.interacaoId)) {
      return {
        sucesso: false,
        erro: "Esta interação já foi resolvida anteriormente.",
        interacaoConcluida: true,
        perfil
      };
    }

    perfil.pontuacao.cuidado += escolha.cuidado || 0;
    perfil.pontuacao.conduta += escolha.conduta || 0;
    perfil.pontuacao.danos += escolha.danos ?? 0;

    perfil.escolhasRealizadas.push(choiceId);

    return {
      sucesso: true,
      mensagem: escolha.mensagem,
      impacto: {
        cuidado: escolha.cuidado,
        conduta: escolha.conduta,
        danos: escolha.danos ?? 0
      },
      perfil
    };
  }
}
```